**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

**SUBJECT NAME: COMPUTER NETWORKS** **SUBJECT CODE: CST52**

Data link layer – design issues – Services - Framing - Error Control - Flow Control - Error detection and correction codes - data link layer protocols -Simplex Protocol – Sliding window Protocols - Medium Access control sub-layer – Channel allocation problem – Multiple Access protocols – ALOHA – CSMA Protocols - Collision-Free Protocols - Limited-Contention Protocols - Wireless LANs - 802.11 Architecture - 802.16 Architecture – Data link layer Switching - Uses of Bridges - Learning Bridges - Spanning Tree Bridges - Repeaters, Hubs, Bridges, Switches, Routers, and Gateways - Virtual LANs.

## 2 MARKS

**1. What is meant by data link layer?**

- Data link layer is most reliable node to node delivery of data. It forms frames from the packets that are received from network layer and gives it to physical layer. It also synchronizes the information which is to be transmitted over the data. Error controlling is easily done. The encoded data are then passed to physical.
- Error detection bits are used by the data link layer. It also corrects the errors. Outgoing messages are assembled into frames. Then the system waits for the acknowledgements to be received after the transmission. It is reliable to send message.

**2. List the functions of data link layer?**

- Framing
- Physical addressing
- Flow control
- Error control
- Access control

**3. What is meant by frames?**

The data link layer divides the stream of bits received from the network layer into manageable data units called frames. The data link layer adds a header to the frame to define the addresses of the sender and receiver of the frame.

**4. List the issues in data link layer?**

The data link layer uses the services of the physical layer to send and receive bits over communication channels. It has a number of functions, including:

1. Providing a well-defined service interface to the network layer.
2. Dealing with transmission errors.
3. Regulating the flow of data so that slow receivers are not swamped by fast senders.

**5. What are the services offered by data link layer?**

The actual services that are offered vary from protocol to protocol. Three reasonable possibilities

1. Unacknowledged connectionless service
2. Acknowledged connectionless service
3. Acknowledged connection-oriented service

**6. Mention the four methods used in Framing?**

1. Byte count
2. Flag bytes with byte stuffing
3. Flag bits with bit stuffing
4. Physical layer coding violations

**7. Define byte stuffing?**

The data link layer on the receiving end removes the escape bytes before giving the data to the network layer. This technique is called byte stuffing.

**8. Define bit stuffing?**

The bit stuffing in which an escape byte is stuffed into the outgoing character stream before a flag byte in the data. It also ensures a minimum density of transitions that help the physical layer maintain synchronization.

**9. What is the use of error control?**

Error control is achieved by adding a trailer at the end of the frame. Duplication of frames are also prevented by using this mechanism. Data Link Layers adds mechanism to prevent duplication of frames.

**10. What is the necessity of flow control? (MAY 2017)**

A flow control mechanism to avoid a fast transmitter from running a slow receiver by buffering the extra bit is provided by flow control. This prevents traffic jam at the receiver side.

**11. Mention the two approaches in flow control mechanism?**

1. **Feedback-based flow control**, the receiver sends back information to the sender giving it permission to send more data, or at least telling the sender how the receiver is doing.
2. **Rate-based flow control**, the protocol has a built-in mechanism that limits the rate at which senders may transmit data, without using feedback from the receiver.

**12. What is Network Interface Cards (NIC)?**

- Each station on an Ethernet network (such as a PC, workstation, or printer) has its own Network Interface Card (NIC). The NIC fits inside the station and provides the station with a 6-byte physical address.
- A Network Interface Card (NIC) is a circuit board or card that is installed in a computer so that it can be connected to a network.
- A network interface card provides the computer with a dedicated, full-time connection to a network. Personal computers and workstations on a local area network (LAN) typically contain a network interface card specifically designed for the LAN transmission technology.
- The physical layer process and some of the data link layer process run on dedicate hardware called a NIC (Network Interface Card).

**13. List the various error correcting codes?**

The four different error-correcting codes are

1. Hamming codes
2. Binary convolutional codes
3. Reed-Solomon codes
4. Low-Density Parity Check codes

**14. List the various codes in error correcting codes?**

1. Block code
2. Systematic code
3. Linear code

**15. Define code-word?**

An n-bit unit containing data and check bits is referred to as an n-bit code-word.

**16. What is hamming codes?**

- In Hamming codes the bits of the code-word are numbered consecutively, starting with bit 1 at the left end, bit 2 to its immediate right, and so on.
- Hamming code is a set of error-correction code s that can be used to detect and correct bit errors that can occur when computer data is moved or stored.

**17. Define Hamming distance? (MAY 2016)**

- The number of bit positions in which two code-words differ is called the Hamming distance.
- The Hamming distance between two words (of the same size) is the number of differences between the corresponding bits.

**18. What is convolutional codes?**

- In a convolutional code, an encoder processes a sequence of input bits and generates a sequence of output bits. There is no natural message size or encoding boundary as in a block code.
- The output depends on the current and previous input bits. That is, the encoder has memory.
- The number of previous bits on which the output depends is called the constraint length of the code.
- Convolutional codes are specified in terms of their rate and constraint length.

**19. Mention the two type of decision decoding?**

1. Soft-decision decoding
2. Hard-decision decoding

**20. What is Reed-Solomon code?**

- Reed-Solomon codes are linear block codes, and they are often systematic too.
- Reed-Solomon codes operate on m bit symbols.
- Reed-Solomon codes are examples of error correcting codes, in which redundant information is added to data so that it can be recovered reliably despite errors in transmission or storage and retrieval.
- The error correction system used on CD's and DVD's is based on a Reed-Solomon code.

**21. Define LDPC?**

- LDPC (Low-Density Parity Check) code codes are linear block codes.
- LDPC code is a method of transmitting a message over a noisy transmission channel.
- In an LDPC code, each output bit is formed from only a fraction of the input bits.

**22. Mention the various error detecting codes?**

The three different error-detecting codes. They are all linear, systematic block codes:

1. Parity
2. Checksums
3. Cyclic Redundancy Checks (CRCs)

**23. What is meant by parity bit?**

- The error-detecting code, in which a single parity bit is appended to the data. The parity bit is chosen so that the number of 1 bits in the code-word is even or odd.
- A parity bit, or check bit, is a bit added to a string of binary code to ensure that the total number of 1-bits in the string is even or odd.

**24. Define checksum?**

- Checksum is often used to mean a group of check bits associated with a message, regardless of how are calculated. A group of parity bits is one example of a checksum.
- A checksum is a simple type of redundancy check that is used to detect errors in data. Errors frequently occur in data when it is written to a disk, transmitted across a network or otherwise manipulated.

**25. What is meant by Cyclic Redundancy Check (CRC)? (NOV 2016)**

- CRC (Cyclic Redundancy Check), also known as a polynomial code.
- Polynomial codes are based upon treating bit strings as representations of polynomials with coefficients of 0 and 1 only.
- A cyclic redundancy check (CRC) is an error-detecting code commonly used in digital networks and storage devices to detect accidental changes to raw data. Blocks of data entering these systems get a short check value attached, based on the remainder of a polynomial division of their contents.

**26. Define Frame header?**

A frame is composed of four fields: **kind, seq, ack,** and **info**, the first three of which contain control information and the last of which may contain actual data to be transferred. These control fields are collectively called the **frame header.**

**27. What is meant by Stop-and-Wait protocol?**

- Protocols in which the sender sends one frame and then waits for an acknowledgement before proceeding are called stop-and-wait.
- Stop-and-Wait Protocol because the sender sends one frame, stops until it receives confirmation from the receiver and then sends the next frame.

**28. What is meant by Automatic Repeat request (ARQ)?**

- Protocols in which the sender waits for a positive acknowledgement before advancing to the next data item are often called ARQ (Automatic Repeat reQuest) or PAR (Positive Acknowledgement with Retransmission).
- Error correction in Stop-and-Wait ARQ is done by keeping a copy of the sent frame and retransmitting of the frame when the timer expires.

**29. Define piggybacking?**

The technique of temporarily delaying outgoing acknowledgements so that they can be hooked onto the next outgoing data frame is known as piggybacking.

A technique called piggybacking is used to improve the efficiency of the bidirectional protocols.

**30. What is sliding window protocol?**

A sliding window protocol is a feature of packet-based data transmission protocols. Sliding window protocols are used where reliable in-order delivery of packets is required, such as in the Data Link Layer (OSI model) as well as in the Transmission Control Protocol (TCP).

**31. What is meant by Go-back-N ARQ protocol?**

- The go-back-n, is for the receiver simply to discard all subsequent frames, sending no acknowledgements for the discarded frames.
- This strategy corresponds to a receive window of size 1.
- Go-Back-N ARQ is a specific instance of the automatic repeat request (ARQ) protocol, in which the sending process continues to send a number of frames specified by a window size even without receiving an acknowledgement (ACK) packet from the receiver.
- It is a special case of the general sliding window protocol with the transmit window size of N and receive window size of 1.
- It can transmit N frames to the peer before requiring an ACK.

**32. What is meant by Selective Repeat ARQ protocol?**

- Selective Repeat is a connection oriented protocol in which both transmitter and receiver have a window of sequence numbers.
- For noisy links, there is another mechanism that does not resend N frames when just one frame is damaged; only the damaged frame is resent. This mechanism is called Selective Repeat ARQ.
- The Selective Repeat Protocol also uses two windows: a send window and a receive window.

**33. Define Medium Access Control (MAC)?**

- The protocols used to determine who goes next on a multi-access channel belong to a sub-layer of the data link layer called the Medium Access Control (MAC) sub-layer.
- The MAC sub-layer is especially important in LANs, particularly wireless ones because wireless is naturally a broadcast channel.

**34. Mention the two types of channel allocation problem?**

1. Static Channel Allocation
2. Dynamic Channel Allocation

**35. List the five assumptions of dynamic channel allocation?**

1. Independent Traffic
2. Single Channel
3. Observable Collisions
4. Continuous or Slotted Time
5. Carrier Sense or No Carrier Sense

**36. Define ALOHA? Mention its versions of ALOHA?**

- Abramson's work, called the ALOHA system, used ground based radio broadcasting, the basic idea is applicable to any system in which uncoordinated users are competing for the use of a single shared channel.
- ALOHA was designed for a radio (wireless) LAN, but it can be used on any shared medium.

The two versions of ALOHA are

1. Pure ALOHA and
2. Slotted ALOHA

**37. Define Pure ALOHA.**

The original ALOHA protocol is called pure ALOHA. This is a simple, but elegant protocol. The idea is that each station sends a frame whenever it has a frame to send. However, since there is only one channel to share, there is the possibility of collision between frames from different stations**.**

**38. Define Slotted ALOHA.**

- The Slotted ALOHA requires that time be segmented into slots of a fixed length exactly equal to the packet transmission time. A packet arriving to be transmitted at any given station must be delayed until the beginning of the next slot.
- In slotted ALOHA we divide the time into slots and force the station to send only at the beginning of the time slot.

**39. Define CSMA protocols?**

Carrier sense multiple access (CSMA) requires that each station first listen to the medium (or check the state of the medium) before sending. In other words, CSMA is based on the principle "sense before transmit" or "listen before talk."

**40. Mention the various types of CSMA?**

The three types of CSMA are

1. 1-persistent CSMA
2. Non-persistent CSMA
3. p-persistent CSMA

**41. What are the difference between collision free and contention based network protocols? (NOV 2015)**

| Collision Free Network Protocols | Contention Based Network Protocols |
|---|---|
| This protocol, known as CSMA/CD (CSMA with Collision Detection), is the basis of the classic Ethernet LAN. | A contention-based protocol (CBP) is a communications protocol for operating wireless telecommunication equipment that allows many users to use the same radio channel without pre-coordination. |
| Collision detection is an analog process. The station's hardware must listen to the channel while it is transmitting. If the signal it reads back is different from the signal it is putting out, it knows that a collision is occurring. | The basic bit-map, token passing and binary count down method are used. |

**42. What is a bit map protocol?**

The collision-free protocol, the basic bit-map method, each contention period consists of exactly N slots. If station 0 has a frame to send, it transmits a 1 bit during the slot 0. No other station is allowed to transmit during this slot.

**43. Define token?**

To pass a small message called a token from one station to the next in the same predefined order. The token represents permission to send. If a station has a frame queued for transmission when it receives the token, it can send that frame before it passes the token to the next station. If it has no queued frame, it simply passes the token.

**44. What is meant by token ring protocol?**

In a token ring protocol, the topology of the network is used to define the order in which stations send. The stations are connected one to the next in a single ring. Passing the token to the next station then simply consists of receiving the token in from one direction and transmitting it out in the other direction.

**45. What is meant by token bus?**

Each station then uses the bus to send the token to the next station in the predefined sequence. Possession of the token allows a station to use the bus to send one frame, as before. This protocol is called token bus.

**46. What is meant by binary count down?**

By using binary station addresses with a channel that combines transmissions. A station wanting to use the channel now broadcasts its address as a binary bit string, starting with the high order bit. All addresses are assumed to be the same length. The bits in each address position from different stations are BOOLEAN ORed together by the channel when they are sent at the same time. We will call this protocol binary countdown.

**47. What is meant by limited-contention protocol?**

The contention and collision-free protocols, arriving at a new protocol that used contention at low load to provide low delay, but used a collision-free technique at high load to provide good channel efficiency. Such protocols, which we will call limited-contention protocols.

**48. What is meant by hidden and exposed terminal problem?**

- The problem of a station not being able to detect a potential competitor for the medium because the competitor is too far away is called the hidden terminal problem.
- The MAC protocol that prevents this kind of deferral from happening because it wastes bandwidth. The problem is called the exposed terminal problem.

**49. Define MACA?**

Multiple Access with Collision Avoidance (MACA) is the basic idea behind it is for the sender to stimulate the receiver into outputting a short frame, so stations nearby can detect this transmission and avoid transmitting for the duration of the upcoming (large) data frame.

**50. What is meant by Access Point (AP)?**

IEEE 802.11 defines the basic service set (BSS) as the building block of a wireless LAN. A basic service set is made of stationary or mobile wireless stations and an optional central base station, known as the access point (AP).

**51. What is meant by distribution system?**

The client sends and receives its packets via the AP. Several access points may be connected together, typically by a wired network called a distribution system.

**52. What is ad hoc network?**

The ad hoc network mode is a collection of computers that are associated so that they can directly send frames to each other. There is no access point.

**53. List the various IEEE 802.11 physical layer?**

| *IEEE* | *Technique* | *Band* | *Modulation* | *Rate (Mbps)* |
|---|---|---|---|---|
| 802.11 | FHSS | 2.4 GHz | FSK | 1 and 2 |
| | DSSS | 2.4 GHz | PSK | 1 and 2 |
| | | Infrared | PPM | 1 and 2 |
| 802.11a | OFDM | 5.725 GHz | PSK or QAM | 6 to 54 |
| 802.11b | DSSS | 2.4 GHz | PSK | 5.5 and 11 |
| 802.11g | OFDM | 2.4 GHz | Different | 22 and 54 |

**54. What is meant by CSMA/CA?**

The Carrier Sense Multiple Access with Collision Avoidance (CSMA/CA) is that a station needs to be able to receive while transmitting to detect a collision. When there is no collision, the station receives one signal: its own signal. When there is a collision, the station receives two signals: its own signal and the signal transmitted by a second station.

**55. What are the two modes in collision avoidance?**

1. The mode of operation is called **DCF (Distributed Coordination Function)** because each station acts independently, without any kind of central control.
2. The standard also includes an optional mode of operation called **PCF (Point Coordination Function)** in which the access point controls all activity in its cell, just like a cellular base station.

**56. What is meant by Network Allocation Vector (NAV)?**

With virtual sensing, each station keeps a logical record of when the channel is in use by tracking the NAV (Network Allocation Vector). Each frame carries a NAV field that says how long the sequence of which this frame is part will take to complete.

When a station sends an RTS frame, it includes the duration of time that it needs to occupy the channel. The stations that are affected by this transmission create a timer called a network allocation vector (NAV) that shows how much time must pass before these stations are allowed to check the channel for idleness.

**57. Mention the various Inter-Frame spacing?**

1. DIFS (DCF Inter Frame Spacing)
2. SIFS (Short Inter Frame Spacing)
3. AIFS (Arbitration Inter Frame Space)
4. EIFS (Extended Inter Frame Spacing)

**58. What is meant by LLC?**

The Data field contains the payload, up to 2312 bytes. The first bytes of this payload are in a format known as LLC (Logical Link Control). This layer is the glue that identifies the higher-layer protocol (e.g., IP) to which the payloads should be passed.

**59. Define OFDMA?**

A flexible scheme for dividing the channel between stations, called OFDMA (Orthogonal Frequency Division Multiple Access). With OFDMA, different sets of subcarriers can be assigned to different stations, so that more than one station can send or receive at once.

**60. Mention the various connecting devices?**

1. Repeaters
2. Hubs
3. Bridges
4. Switches
5. Routers and
6. Gateways

**61. What is the uses of bridges?**

- First, many university and corporate departments have their own LANs to connect their own personal computers, servers, and devices such as printers.
- Second, the organization may be geographically spread over several buildings separated by considerable distances.
- Third, it may be necessary to split what is logically a single LAN into separate LANs (connected by bridges) to accommodate the load.

**62. What is meant by cut-through switching?**

The design reduces the latency of passing through the bridge, as well as the number of frames that the bridge must be able to buffer. It is referred to as cut-through switching or wormhole routing and is usually handled in hardware.

**63. Define spanning tree?** **(MAY 2016)**

In graph theory, a spanning tree is a graph in which there is no loop. In a bridged LAN, this means creating a topology in which each LAN can be reached from any other LAN through one path only (no loop).

We cannot change the physical topology of the system because of physical connections between cables and bridges, but we can create a logical topology that overlays the physical one.

**64. What is meant by Repeaters?**

- A repeater is a device that operates only in the physical layer.
- Signals that carry information within a network can travel a fixed distance before attenuation endangers the integrity of the data.
- A repeater receives a signal and, before it becomes too weak or corrupted, regenerates the original bit pattern.
- The repeater then sends the refreshed signal.
- A repeater can extend the physical length of a LAN.

**65. What is meant by hub?** **(MAY, NOV 2016)**

- A hub has a number of input lines that it joins electrically.
- Frames arriving on any of the lines are sent out on all the others.
- If two frames arrive at the same time, they will collide, just as on a coaxial cable.
- All the lines coming into a hub must operate at the same speed.
- Hubs differ from repeaters in that they do not (usually) amplify the incoming signals and are designed for multiple input lines.

**66. What is meant by bridges?**

- A bridge operates in both the physical and the data link layer.
- As a physical layer device, it regenerates the signal it receives.
- As a data link layer device, the bridge can check the physical (MAC) addresses (source and destination) contained in the frame.
- A common example is a bridge with ports that connect to 10-, 100-, and 1000-Mbps Ethernet.

**67. What is meant by Switches?** **(NOV 2016)**

- A switch may refer to an Ethernet switch or a completely different kind of device that makes forwarding decisions, such as a telephone switch.

- Switch is data link layer device. Switch can perform error checking before forwarding data that makes it very efficient as it does not forward packets that have errors and forward good packets selectively to correct port only.

**68. What is meant by routers?**

- A router is a three-layer device that routes packets based on their logical addresses (host-to-host addressing).
- A router normally connects LANs and WANs in the Internet and has a routing table that is used for making decisions about the route.
- The routing tables are normally dynamic and are updated using routing protocols.
- For an IP packet, the packet header will contain a 32-bit (IPv4) or 128-bit (IPv6) address.

**69. What is meant by gateway?**

- A gateway is a passage to connect two networks together that may work upon different networking models.
- They basically works as the messenger agents that take data from one system, interpret it, and transfer it to another system.
- Gateways are also called protocol converters and can operate at any network layer.
- Gateways are generally more complex than switch or router.

**70. What are Virtual LANs? (NOV 2015)**

- A Virtual Local Area Network (VLAN) as a local area network configured by software, not by physical wiring.
- In response to customer requests for more flexibility, network vendors began working on a way to rewire buildings entirely in software. The resulting concept is called a VLAN (Virtual LAN).

**71. Differentiate between physical and logical address. (MAY 2017)**

- The physical address, also known as the link address, is the address of a node as defined by its LAN or WAN. It is included in the frame used by the data link layer. It is the lowest-level address. The physical addresses have authority over the network (LAN or WAN).
- Logical addresses are necessary for universal communications that are independent of underlying physical networks. A logical address in the Internet is currently a 32-bit address that can uniquely define a host connected to the Internet. No two publicly addressed and visible hosts on the Internet can have the same IP address.

## 11 MARKS

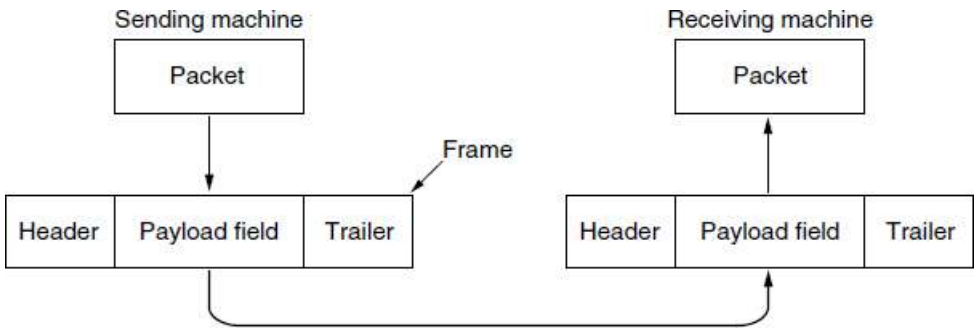
**1. Explain in detail about design issues in data link layer? (11 MARKS)**

The data link layer uses the services of the physical layer to send and receive bits over communication channels.

It has a number of functions, including:

1. Providing a well-defined service interface to the network layer.
2. Dealing with transmission errors.
3. Regulating the flow of data so that slow receivers are not swamped by fast senders.

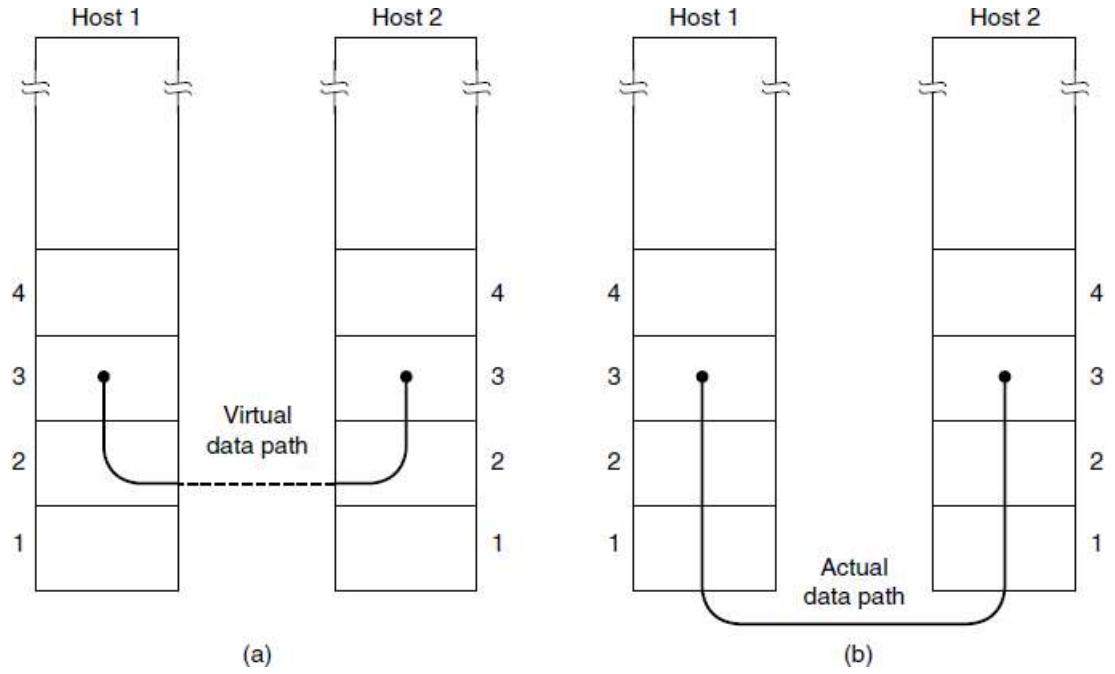
To accomplish these goals, the data link layer takes the packets it gets from the network layer and encapsulates them into **frames** for transmission. Each frame contains a frame header, a payload field for holding the packet, and a frame trailer, as illustrated in Figure. Frame management forms the heart of what the data link layer does.




**Relationship between packets and frames**

**1. Services Provided to the Network Layer**

- The function of the data link layer is to provide services to the network layer.
- The principal service is transferring data from the network layer on the source machine to the network layer on the destination machine.
- On the source machine is an entity, call it a process, in the network layer that hands some bits to the data link layer for transmission to the destination.
- The job of the data link layer is to transmit the bits to the destination machine so they can be handed over to the network layer there, as shown in Figure (a).
- The actual transmission follows the path of Figure (b), but it is easier to think in terms of two data link layer processes communicating using a data link protocol.




**(a) Virtual communication** **(b) Actual communication**

The data link layer can be designed to offer various services. The actual services that are offered vary from protocol to protocol.

Three reasonable possibilities are

1. Unacknowledged connectionless service
2. Acknowledged connectionless service
3. Acknowledged connection-oriented service

- ***Unacknowledged connectionless service*** consists of having the source machine send independent frames to the destination machine without having the destination machine acknowledge them. Ethernet is a good example of a data link layer that provides this class of service. No logical connection is established beforehand or released afterward. If a frame is lost due to noise on the line, no attempt is made to detect the loss or recover from it in the data link layer. This class of service is appropriate when the error rate is very low, so recovery is left to higher layers. It is also appropriate for real-time traffic, such as voice, in which late data are worse than bad data.
- The next step up in terms of reliability is ***acknowledged connectionless service***. When this service is offered, there are still no logical connections used, but each frame sent is individually acknowledged. In this way, the sender knows whether a frame has arrived correctly or been lost. If it has not arrived within a specified time interval, it can be sent again. This service is useful over unreliable channels, such as wireless systems. 802.11 (WiFi) is a good example of this class of service.
- Getting back to our services, the most sophisticated service the data link layer can provide to the network layer is ***connection-oriented service***. With this service, the source and destination machines establish a connection before any data are transferred. Each frame sent over the connection is numbered, and the data link layer guarantees that each frame sent is indeed received. Furthermore, it

guarantees that each frame is received exactly once and that all frames are received in the right order. Connection-oriented service thus provides the network layer processes with the equivalent of a reliable bit stream. It is appropriate over long, unreliable links such as a satellite channel or a long-distance telephone circuit. If acknowledged connectionless service were used, it is conceivable that lost acknowledgements could cause a frame to be sent and received several times, wasting bandwidth.

When connection-oriented service is used, transfers go through three distinct phases.

1. In the first phase, the connection is established by having both sides initialize variables and counters needed to keep track of which frames have been received and which ones have not.
2. In the second phase, one or more frames are actually transmitted.
3. In the third and final phase, the connection is released, freeing up the variables, buffers, and other resources used to maintain the connection.

**2. Framing**

- The data link layer to break up the bit stream into discrete frames, compute a short token called a **checksum** for each frame, and include the checksum in the frame when it is transmitted.
- When a frame arrives at the destination, the checksum is recomputed.
- If the newly computed checksum is different from the one contained in the frame, the data link layer knows that an error has occurred and takes steps to deal with it (e.g., discarding the bad frame and possibly also sending back an error report).

Breaking up the bit stream into frames is more difficult than it at first appears. A good design must make it easy for a receiver to find the start of new frames while using little of the channel bandwidth.

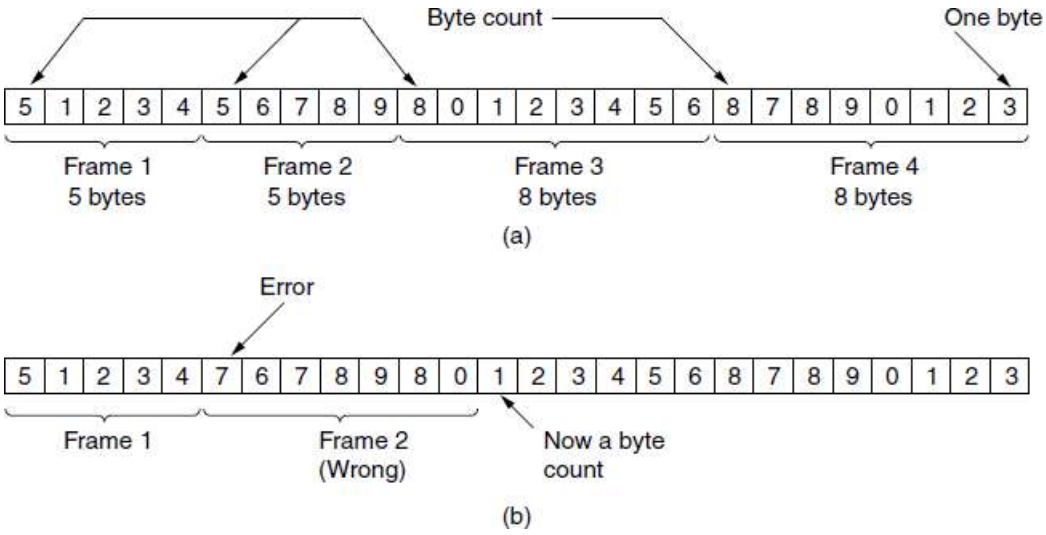
The four methods:

1. Byte count
2. Flag bytes with byte stuffing
3. Flag bits with bit stuffing
4. Physical layer coding violations

The **first framing** method uses a field in the header to specify the number of bytes in the frame. When the data link layer at the destination sees the byte count, it knows how many bytes follow and hence where the end of the frame is. This technique is shown in Figure (a) for four small example frames of sizes 5, 5, 8, and 8 bytes, respectively.

The trouble with this algorithm is that the count can be garbled by a transmission error. For example, if the byte count of 5 in the second frame of Figure (b) becomes a 7 due to a single bit flip, the destination will get out of synchronization. It will then be unable to locate the correct start of the next frame. Even if the checksum is incorrect so the destination knows that the frame is bad, it still has no way of telling where the next frame starts. Sending a frame back to the source asking for a retransmission does not

help either, since the destination does not know how many bytes to skip over to get to the start of the retransmission. For this reason, the byte count method is rarely used by itself.



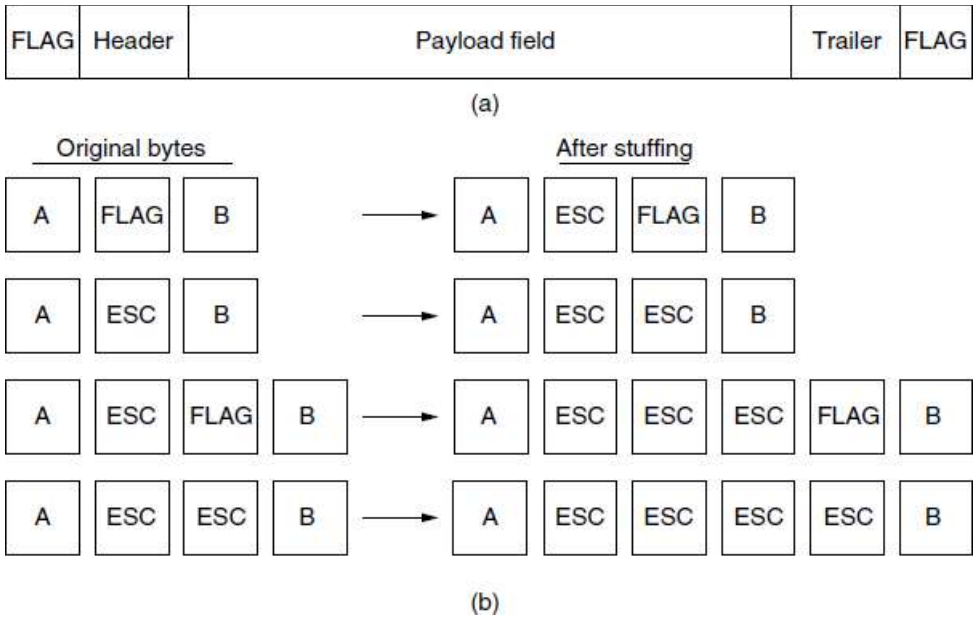

**A byte stream. (a) Without errors. (b) With one error.**

The **second framing** method gets around the problem of resynchronization after an error by having each frame start and end with special bytes. Often the same byte, called a **flag byte**, is used as both the starting and ending delimiter. This byte is shown in Figure (a) as FLAG. Two consecutive flag bytes indicate the end of one frame and the start of the next. Thus, if the receiver ever loses synchronization it can just search for two flag bytes to find the end of the current frame and the start of the next frame.

However, there is a still a problem we have to solve. It may happen that the flag byte occurs in the data, especially when binary data such as photographs or songs are being transmitted. This situation would interfere with the framing. One way to solve this problem is to have the sender's data link layer insert a special escape byte (ESC) just before each "accidental" flag byte in the data. Thus, a framing flag byte can be distinguished from one in the data by the absence or presence of an escape byte before it. The data link layer on the receiving end removes the escape bytes before giving the data to the network layer. This technique is called **byte stuffing.**

Of course, the next question is: what happens if an escape byte occurs in the middle of the data? The answer is that it, too, is stuffed with an escape byte. At the receiver, the first escape byte is removed, leaving the data byte that follows it. Some examples are shown in Figure (b). In all cases, the byte sequence delivered after de-stuffing is exactly the same as the original byte sequence. We can still search for a frame boundary by looking for two flag bytes in a row, without bothering to undo escapes.

The byte-stuffing scheme depicted in Figure is a slight simplification of the one used in **PPP (Point-to-Point Protocol)**, which is used to carry packets over communications links.



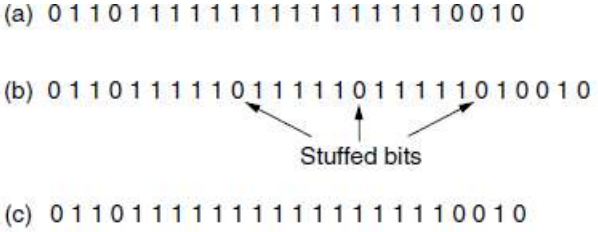

**(a) A frame delimited by flag bytes. (b) Four examples of byte sequences before and after byte stuffing.**

The **third method** of delimiting the bit stream gets around a disadvantage of byte stuffing, which is that it is tied to the use of 8-bit bytes. Framing can be also be done at the bit level, so frames can contain an arbitrary number of bits made up of units of any size. It was developed for the once very popular **HDLC (High level Data Link Control)** protocol.

Each frame begins and ends with a special bit pattern, 01111110 or 0x7E in hexadecimal. This pattern is a flag byte. Whenever the sender's data link layer encounters five consecutive 1s in the data, it automatically stuffs a 0 bit into the outgoing bit stream.

This **bit stuffing** is analogous to byte stuffing, in which an escape byte is stuffed into the outgoing character stream before a flag byte in the data. It also ensures a minimum density of transitions that help the physical layer maintain synchronization. USB (Universal Serial Bus) uses bit stuffing for this reason.

When the receiver sees five consecutive incoming 1 bits, followed by a 0 bit, it automatically de-stuffs (i.e., deletes) the 0 bit. Byte stuffing is completely transparent to the network layer in both computers, so is bit stuffing. If the user data contain the flag pattern, 01111110, this flag is transmitted as 011111010 but stored in the receiver's memory as 01111110. Figure gives an example of bit stuffing.




Bit stuffing. (a) The original data. (b) The data as they appear on the line. (c) The data as they are stored in the receiver's memory after destuffing.

The **last method** of framing is to use a shortcut from the physical layer. The encoding of bits as signals often includes redundancy to help the receiver. This redundancy means that some signals will not occur in regular data.

For example, in the 4B/5B line code 4 data bits are mapped to 5 signal bits to ensure sufficient bit transitions. This means that 16 out of the 32 signal possibilities are not used. We can use some reserved signals to indicate the start and end of frames. In effect, we are using ‘‘coding violations’’ to delimit frames.

## 3. Error Control

Having solved the problem of marking the start and end of each frame, we come to the next problem: how to make sure all frames are eventually delivered to the network layer at the destination and in the proper order. Assume for the moment that the receiver can tell whether a frame that it receives contains correct or faulty information.

For unacknowledged connectionless service it might be fine if the sender just kept outputting frames without regard to whether they were arriving properly. But for reliable, connection-oriented service it would not be fine at all.

The usual way to ensure reliable delivery is to provide the sender with some feedback about what is happening at the other end of the line. Typically, the protocol calls for the receiver to send back special control frames bearing positive or negative acknowledgements about the incoming frames. If the sender receives a positive acknowledgement about a frame, it knows the frame has arrived safely. On the other hand, a negative acknowledgement means that something has gone wrong and the frame must be transmitted again.

An additional complication comes from the possibility that hardware troubles may cause a frame to vanish completely (e.g., in a noise burst). In this case, the receiver will not react at all, since it has no reason to react. Similarly, if the acknowledgement frame is lost, the sender will not know how to proceed. It should be clear that a protocol in which the sender transmits a frame and then waits for an acknowledgement, positive or negative, will hang forever if a frame is ever lost due to, for example, malfunctioning hardware or a faulty communication channel.

This possibility is dealt with by introducing timers into the data link layer. When the sender transmits a frame, it generally also starts a timer. The timer is set to expire after an interval long enough for the frame to reach the destination, be processed there, and have the acknowledgement propagate back to the sender. Normally, the frame will be correctly received and the acknowledgement will get back before the timer runs out, in which case the timer will be canceled.

However, if either the frame or the acknowledgement is lost, the timer will go off, alerting the sender to a potential problem. The obvious solution is to just transmit the frame again. However, when frames may be transmitted multiple times there is a danger that the receiver will accept the same frame two or more times and pass it to the network layer more than once. To prevent this from happening, it is

generally necessary to assign sequence numbers to outgoing frames, so that the receiver can distinguish retransmissions from originals.

The whole issue of managing the timers and sequence numbers so as to ensure that each frame is ultimately passed to the network layer at the destination exactly once, no more and no less, is an important part of the duties of the data link layer.

**4. Flow Control**

Another important design issue that occurs in the data link layer is what to do with a sender that systematically wants to transmit frames faster than the receiver can accept them. This situation can occur when the sender is running on a fast, powerful computer and the receiver is running on a slow, low-end machine.

A common situation is when a smart phone requests a Web page from a far more powerful server, which then turns on the fire hose and blasts the data at the poor helpless phone until it is completely swamped. Even if the transmission is error free, the receiver may be unable to handle the frames as fast as they arrive and will lose some.

Two approaches are commonly used.

1. In the first one, **feedback-based flow control**, the receiver sends back information to the sender giving it permission to send more data, or at least telling the sender how the receiver is doing.
2. In the second one, **rate-based flow control**, the protocol has a built-in mechanism that limits the rate at which senders may transmit data, without using feedback from the receiver.

**2. Explain in detail about Error Detection and Error Correction codes? (11 MARKS) (NOV 2015) (MAY 2016, 2017)**

Data can be corrupted or error may occur during transmission. Network designers have developed two basic strategies for dealing with errors. Both add redundant information to the data that is sent.

One strategy is to include enough redundant information to enable the receiver to deduce what the transmitted data must have been. The other is to include only enough redundancy to allow the receiver to deduce that an error has occurred (but not which error) and have it request a retransmission. The former strategy uses **error-correcting codes** and the latter uses **error-detecting codes**. The use of error-correcting codes is often referred to as **FEC** (**Forward Error Correction**).

**Types of Errors**

Whenever bits flow from one point to another, they are subject to unpredictable changes because of interference. This interference can change the shape of the signal. In a single-bit error, a 0 is changed to a 1 or a 1 to a 0. In a burst error, multiple bits are changed.

For example, a 11100 s burst of impulse noise on a transmission with a data rate of 1200 bps might change all or some of the 12 bits of information.

**Single-Bit Error**

The term single-bit error means that only 1 bit of a given data unit (such as a byte, character, or packet) is changed from 1 to 0 or from 0 to 1.

**Burst Error**

The term burst error means that 2 or more bits in the data unit have changed from 1 to 0 or from 0 to 1.

## ERROR-CORRECTING CODES

The four different error-correcting codes are

1. **Hamming codes**
2. **Binary convolutional codes**
3. **Reed-Solomon codes**
4. **Low-Density Parity Check codes**

### 1. Hamming codes

- All of these codes add redundancy to the information that is sent.
- A frame consists of m data (i.e., message) bits and r redundant (i.e. check) bits.
- In a **block code**, the r check bits are computed solely as a function of the m data bits with which they are associated, as though the m bits were looked up in a large table to find their corresponding r check bits.
- In a **systematic code**, the m data bits are sent directly, along with the check bits, rather than being encoded themselves before they are sent.
- In a **linear code**, the r check bits are computed as a linear function of the m data bits.
- **Exclusive OR (XOR)** or **modulo 2 addition** is a popular choice. This means that encoding can be done with operations such as matrix multiplications or simple logic circuits.

- Let the total length of a block be n (i.e., n = m + r).
- We will describe this as an (n, m) code.
- An n-bit unit containing data and check bits is referred to as an n-bit **code word**.
- The **code rate**, or simply rate, is the fraction of the code word that carries information that is not redundant, or m/n.
- The rates used in practice vary widely. They might be 1/2 for a noisy channel, in which case half of the received information is redundant, or close to 1 for a high-quality channel, with only a small number of check bits added to a large message.

To understand how errors can be handled, it is necessary to first look closely at what an error really is. Given any two code words that may be transmitted or received—say, 10001001 and 10110001—it is possible to determine how many corresponding bits differ. In this case, 3 bits differ.

To determine how many bits differ, just XOR the two code words and count the number of 1 bits in the result.

For example:

10001001
10110001
00111000

The number of bit positions in which two code words differ is called the **Hamming distance**. Its significance is that if two code words are a Hamming distance d apart, it will require d single-bit errors to convert one into the other.

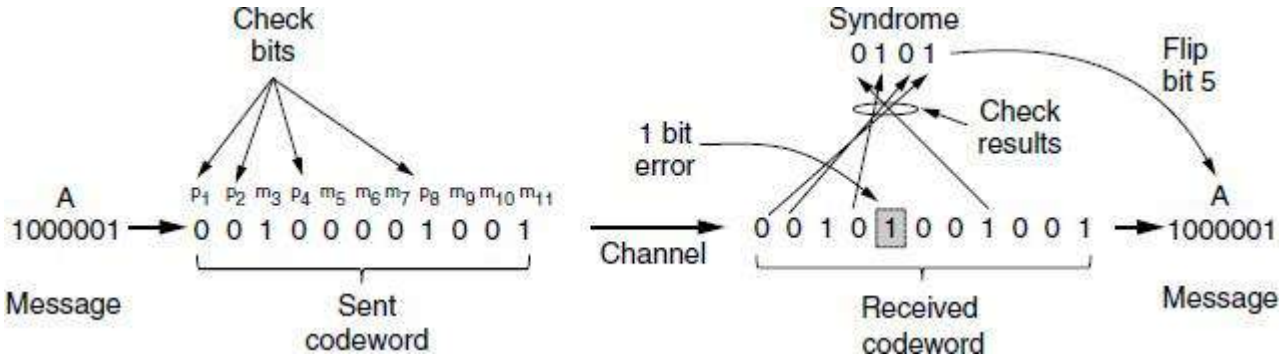
Given the algorithm for computing the check bits, it is possible to construct a complete list of the legal code words, and from this list to find the two code words with the smallest Hamming distance. This distance is the Hamming distance of the complete code.

Using n = m + r, this requirement becomes

$$(m + r + 1) \leq 2^r$$

Given m, this puts a lower limit on the number of check bits needed to correct single errors.

In **Hamming codes** the bits of the code word are numbered consecutively, starting with bit 1 at the left end, bit 2 to its immediate right, and so on. The bits that are powers of 2 (1, 2, 4, 8, 16, etc.) are check bits. The rest (3, 5, 6, 7, 9, etc.) are filled up with the m data bits. This pattern is shown for an (11, 7) Hamming code with 7 data bits and 4 check bits in Figure.




**Example of an (11, 7) Hamming code correcting a single-bit error**

- Each check bit forces the modulo 2 sum, or parity, of some collection of bits, including itself, to be even (or odd). A bit may be included in several check bit computations.
- To see which check bits the data bit in position k contributes to, rewrite k as a sum of powers of 2.
- For example, 11 = 1 + 2 + 8 and 29 = 1 + 4 + 8 + 16. A bit is checked by just those check bits occurring in its expansion (e.g., bit 11 is checked by bits 1, 2, and 8).
- In the example, the check bits are computed for even parity sums for a message that is the ASCII letter "A."

This construction gives a code with a Hamming distance of 3, which means that it can correct single errors (or detect double errors). The reason for the very careful numbering of message and check bits becomes apparent in the decoding process. When a code word arrives, the receiver redoes the check bit computations including the values of the received check bits. We call these the check results.

If the check bits are correct then, for even parity sums, each check result should be zero. In this case the code word is accepted as valid.

If the check results are not all zero, however, an error has been detected. The set of check results forms the error syndrome that is used to pinpoint and correct the error. In Figure, a single-bit error occurred on the channel so the check results are 0, 1, 0, and 1 for k = 8, 4, 2, and 1, respectively. This gives a s**yndrome** of 0101 or 4 + 1=5.

By the design of the scheme, this means that the fifth bit is in error. Flipping the incorrect bit (which might be a check bit or a data bit) and discarding the check bits gives the correct message of an ASCII "A."

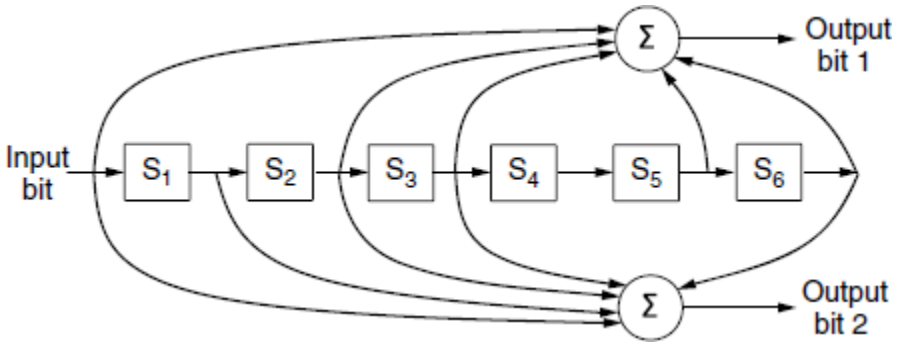
## 2. Binary convolutional codes

Hamming distances are valuable for understanding block codes, and Hamming codes are used in error-correcting memory. However, most networks use stronger codes. The second code we will look at is a **convolutional code**.

This code is the only one we will cover that is not a block code. In a convolutional code, an encoder processes a sequence of input bits and generates a sequence of output bits. There is no natural message size or encoding boundary as in a block code. The output depends on the current and previous input bits. That is, the encoder has memory. The number of previous bits on which the output depends is called the **constraint length** of the code.

Convolutional codes are specified in terms of their rate and constraint length. Convolutional codes are widely used in deployed networks, for example, as part of the GSM mobile phone system, in satellite communications, and in 802.11. As an example, a popular convolutional code is shown in Fig. 3-7. This code is known as the NASA convolutional code of $r = 1/2$ and $k = 7$, since it was first used for the Voyager space missions starting in 1977. Since then it has been liberally reused, for example, as part of 802.11.

As an example, a popular convolutional code is shown in Figure.



**The NASA binary convolutional code used in 802.11**

### 3. Reed-Solomon codes

The third kind of error-correcting code we will describe is the **Reed-Solomon code**. Like Hamming codes, Reed-Solomon codes are linear block codes, and they are often systematic too. Unlike Hamming codes, which operate on individual bits, Reed-Solomon codes operate on *m* bit symbols. Naturally, the mathematics are more involved, so we will describe their operation by analogy.

Reed-Solomon codes are based on the fact that every *n* degree polynomial is uniquely determined by $n + 1$ points.

For example, a line having the form $ax + b$ is determined by two points. Extra points on the same line are redundant, which is helpful for error correction. Imagine that we have two data points that represent a ine and we send those two data points plus two check points chosen to lie on the same line. If one of the points is received in error, we can still recover the data points by fitting a line to the received points. Three of the points will lie on the line, and one point, the one in error, will not. By finding the line we have corrected the error.

Reed-Solomon codes are actually defined as polynomials that operate over finite fields, but they work in a similar manner. For m bit symbols, the code words are $2^m - 1$ symbols long.

### 4. Low-Density Parity Check codes

An LDPC (Low-Density Parity Check) code, each output bit is formed from only a fraction of the input bits. This leads to a matrix representation of the code that has a low density of 1s, hence the name for the code. The received code words are decoded with an approximation algorithm that iteratively improves on a best fit of the received data to a legal code word. This corrects errors.

LDPC codes are practical for large block sizes and have excellent error-correction abilities that outperform many other codes.

They are part of the standard for digital video broadcasting, 10 Gbps Ethernet, power-line networks, and the latest version of 802.11.

## Error-Detecting Codes

In error detection, the receiver needs to know only that the received code word is invalid; in error correction the receiver needs to find (or guess) the original code word sent. The more redundant bits for error correction than for error detection.

Error-correcting codes are widely used on wireless links, which are notoriously noisy and error prone when compared to optical fibers. However, over fiber or high-quality copper, the error rate is much lower, so error detection and retransmission is usually more efficient there for dealing with the occasional error. Three different error-detecting codes. They are all linear, systematic block codes:

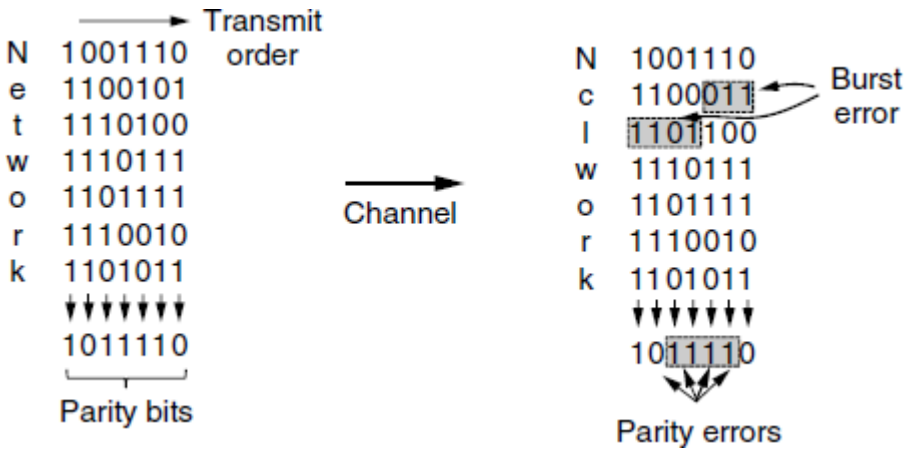
1. **Parity**
2. **Checksums**
3. **Cyclic Redundancy Checks (CRCs)**

## 1. Parity bit

- The first error-detecting code, in which a single parity bit is appended to the data.
- The parity bit is chosen so that the number of 1 bits in the code word is even (or odd).
- This is equivalent to computing the (even) parity bit as the modulo 2 sum or XOR of the data bits.
- For example, when 1011010 is sent in even parity, a bit is added to the end to make it 10110100. With odd parity 1011010 becomes 10110101.
- A code with a single parity bit has a distance of 2, since any single-bit error produces a code word with the wrong parity. This means that it can detect single-bit errors.

We provide a better protection against burst errors: we can compute the parity bits over the data in a different order than the order in which the data bits are transmitted. Doing so is called **interleaving**.

In this case, we will compute a parity bit for each of the n columns and send all the data bits as k rows, sending the rows from top to bottom and the bits in each row from left to right in the usual manner. At the last row, we send the n parity bits. This transmission order is shown in Figure for n = 7 and k = 7.




**Interleaving of parity bits to detect a burst error**

- Interleaving is a general technique to convert a code that detects (or corrects) isolated errors into a code that detects (or corrects) burst errors.
- In Figure, when a burst error of length n = 7 occurs, the bits that are in error are spread across different columns.
- At most 1 bit in each of the n columns will be affected, so the parity bits on those columns will detect the error. This method uses n parity bits on blocks of kn data bits to detect a single burst error of length n or less.
- A burst of length n + 1 will pass undetected, however, if the first bit is inverted, the last bit is inverted, and all the other bits are correct.

## 2. Checksums

- The second kind of error-detecting code, the **checksum**, is closely related to groups of parity bits.

- The "checksum" is often used to mean a group of check bits associated with a message, regardless of how are calculated.
- A **group of parity bits** is one example of a checksum.
- However, the stronger checksums based on a running sum of the data bits of the message.
- The checksum is usually placed at the end of the message, as the complement of the sum function.
- The errors may be detected by summing the entire received code word, both data bits and checksum.
- If the result comes out to be zero, no error has been detected.

One example of a checksum is the 16-bit Internet checksum used on all Internet packets as part of the IP protocol. This checksum is a sum of the message bits divided into 16-bit words. This method operates on words rather than on bits, as in parity, errors that leave the parity unchanged can still alter the sum and be detected.

For example, if the lowest order bit in two different words is flipped from a 0 to a 1, a parity check across these bits would fail to detect an error. However, two 1s will be added to the 16-bit checksum to produce a different result. The error can then be detected. A **Fletcher's checksum** includes a positional component, adding the product of the data and its position to the running sum. This provides stronger detection of changes in the position of data.

**3. Cyclic Redundancy Checks (CRCs)**

- The third error-detecting code is in widespread use at the link layer: the **CRC (Cyclic Redundancy Check),** also known as a **polynomial code**.
- Polynomial codes are based upon treating bit strings as representations of polynomials with coefficients of 0 and 1 only.
- A k-bit frame is regarded as the coefficient list for a polynomial with k terms, ranging from $x^{k-1}$ to $x^0$. Such a polynomial is said to be of degree k − 1.
- The high-order (leftmost) bit is the coefficient of $x^{k-1}$, the next bit is the coefficient of $x^{k-2}$, and so on.

For example, 110001 has 6 bits and thus represents a six-term polynomial with coefficients 1, 1, 0, 0, 0, and 1: $1x^5 + 1x^4 + 0x^3 + 0x^2 + 0x^1 + 1x^0$.

Polynomial arithmetic is done modulo 2, according to the rules of algebraic field theory. It does not have carries for addition or borrows for subtraction. Both addition and subtraction are identical to exclusive OR.

For example:

| | | | |
|---|---|---|---|
| 10011011 | 00110011 | 11110000 | 01010101 |
| + 11001010 | + 11001101 | − 10100110 | − 10101111 |
| 01010001 | 11111110 | 01010110 | 11111010 |

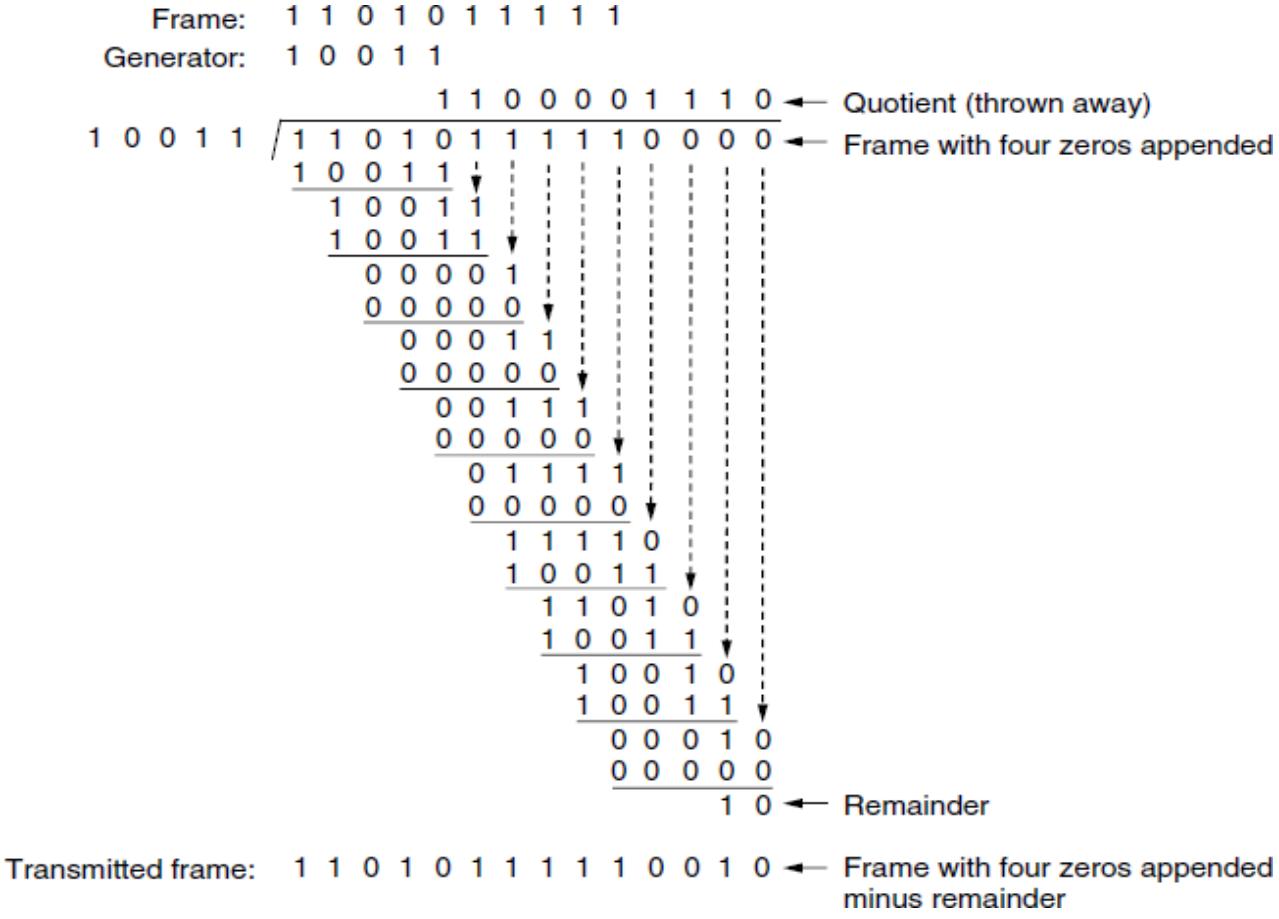
Long division is carried out in exactly the same way as it is in binary except that the subtraction is again done modulo 2. A divisor is said "to go into" a dividend if the dividend has as many bits as the divisor.

- When the polynomial code method is employed, the sender and receiver must agree upon a **generator polynomial,** G(x).
- Both the high- and low-order bits of the generator must be 1.
- To compute the CRC for some frame with m bits corresponding to the polynomial M(x), the frame must be longer than the generator polynomial.
- The idea is to append a CRC to the end of the frame in such a way that the polynomial represented by the check summed frame is divisible by G(x).
- When the receiver gets the check summed frame, it tries dividing it by G(x).
- If there is a remainder, there has been a transmission error.

The algorithm for computing the CRC is as follows:

1. Let r be the degree of G(x). Append r zero bits to the low-order end of the frame so it now contains m + r bits and corresponds to the polynomial $x^r M(x)$.
2. Divide the bit string corresponding to G(x) into the bit string corresponding to $x^r M(x)$, using modulo 2 division.
3. Subtract the remainder from the bit string corresponding to $x^r M(x)$ using modulo 2 subtraction. The result is the check summed frame to be transmitted. Call its polynomial T(x).

   Figure illustrates the calculation for a frame 1101011111 using the generator $G(x) = x^4 + x + 1$.




**Example calculation of the CRC**

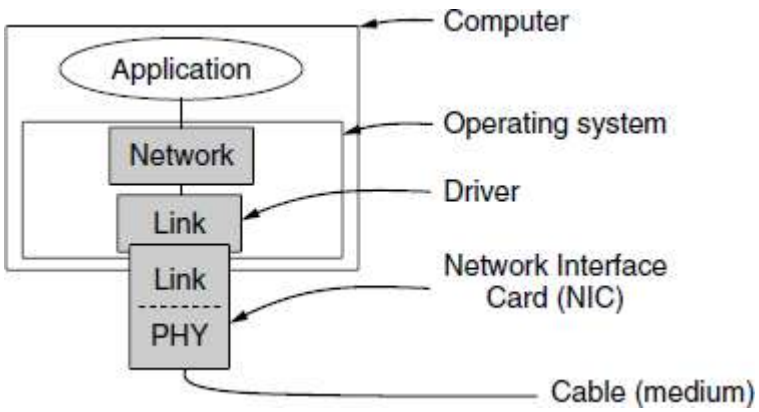
**3. Explain in detail about elementary data link layer protocols? (6 MARKS)**

The two main functions of the data link layer are data link control and media access control.

1. The first, data link control, deals with the design and procedures for communication between two adjacent nodes: node-to-node communication.
2. The second function of the data link layer is media access control, or how to share the link.

**Data link control** functions include framing, flow and error control, and software implemented protocols that provide smooth and reliable transmission of frames between nodes.

To start with, we assume that the physical layer, data link layer, and network layer are independent processes that communicate by passing messages back and forth. A common implementation is shown in Figure.




**Implementation of the physical, data link, and network layers**

- The physical layer process and some of the data link layer process run on dedicate hardware called a **NIC (Network Interface Card)**.
- The rest of the link layer process and the network layer process run on the main CPU as part of the operating system, with the software for the link layer process often taking the form of a **device driver.**
- However, other implementations are also possible (e.g., three processes offloaded to dedicated hardware called a **network accelerator**, or three processes running on the main CPU on a software-defined ratio).
- Actually, the preferred implementation changes from decade to decade with technology trade-offs.
- In any event, the three layers as separate processes makes the discussion conceptually cleaner and also serves to emphasize the independence of the layers.

Another key assumption is that machine A wants to send a long stream of data to machine B, using a reliable, connection-oriented service. Consider the case where B also wants to send data to A simultaneously. A is assumed to have an infinite supply of data ready to send and never has to wait for data to be produced. Instead, when A's data link layer asks for data, the network layer is always able to comply immediately.

We also assume that machines do not crash. That is, these protocols deal with communication errors, but not the problems caused by computers crashing and rebooting.

The data link layer is concerned, the packet passed across the interface to it from the network layer is pure data, whose every bit is to be delivered to the destination's network layer. The fact that the destination's network layer may interpret part of the packet as a header is of no concern to the data link layer.

When the data link layer accepts a packet, it encapsulates the packet in a frame by adding a data link header and trailer to it. Thus, a frame consists of an embedded packet, some control information (in the header), and a checksum (in the trailer). The frame is then transmitted to the data link layer on the other machine. We will assume that there exist suitable library procedures to physical layer to send a frame and from physical layer to receive a frame. These procedures compute and append or check the checksum (which is usually done in hardware).

```
#define MAX_PKT 1024                                    /* determines packet size in bytes */
typedef enum {false, true} boolean;                     /* boolean type */
typedef unsigned int seq_nr;                            /* sequence or ack numbers */
typedef struct {unsigned char data[MAX_PKT];} packet;   /* packet definition */
typedef enum {data, ack, nak} frame_kind;               /* frame kind definition */
typedef struct {                                        /* frames are transported in this layer */
    frame_kind_kind;                                    /* what kind of frame is it? */
    seq_nr_seq;                                         /* sequence number */
    seq_nr_ack;                                         /* acknowledgement number */
    packet_info;                                        /* the network layer packet */
} frame;

/* Wait for an event to happen; return its type in event. */
void wait_for_event(event_type *event);

/* Fetch a packet from the network layer for transmission on the channel. */
void from_network_layer(packet *p);

/* Deliver information from an inbound frame to the network layer. */
void to_network_layer(packet *p);

/* Go get an inbound frame from the physical layer and copy it to r. */
void from_physical_layer(frame *r);

/* Pass the frame to the physical layer for transmission. */
void to_physical_layer(frame *s);
/* Start the clock running and enable the timeout event. */
void start_timer(seq_nr k);
```

```
/* Stop the clock and disable the timeout event. */
void stop_timer(seq_nr k);

/* Start an auxiliary timer and enable the ack timeout event. */
void start_ack_timer(void);

/* Stop the auxiliary timer and disable the ack timeout event. */
void stop_ack_timer(void);

/* Allow the network layer to cause a network layer ready event. */
void enable_network_layer(void);

/* Forbid the network layer from causing a network layer ready event. */
void disable_network_layer(void);

/* Macro inc is expanded in-line: increment k circularly. */
#define inc(k) if (k < MAX_SEQ) k = k + 1; else k = 0
```

**Some definitions needed in the protocols to follow. These definitions are located in the file protocol.h.**

Five data structures are defined there:

1. boolean,
2. seq_nr,
3. packet,
4. frame_kind, and
5. frame

- A **boolean** is an enumerated type and can take on the values true and false.
- A **seq_nr** is a small integer used to number the frames so that we can tell them apart. These sequence numbers run from 0 up to and including MAX _SEQ, which is defined in each protocol needing it.
- A **packet** is the unit of information exchanged between the network layer and the data link layer on the same machine, or between network layer peers.
- A frame is composed of four fields: **kind, seq, ack**, and **info**, the first three of which contain control information and the last of which may contain actual data to be transferred. These control fields are collectively called the **frame header.**

- The **kind field** tells whether there are any data in the frame, because some of the protocols distinguish frames containing only control information from those containing data as well.
- The **seq** and **ack fields** are used for sequence numbers and acknowledgements, respectively.
- The **info field** of a data frame contains a single packet.

**4. Explain in detail about simplex protocol in data link layer? (5 MARKS)**

**A Utopian Simplex Protocol**

The protocol consists of two distinct procedures, a sender and a receiver. The sender runs in the data link layer of the source machine, and the receiver runs in the data link layer of the destination machine. No sequence numbers or acknowledgements are used here, so MAX_SEQ is not needed. The only event type possible is frame_arrival (i.e., the arrival of an undamaged frame).

The **sender** is in an infinite while loop just pumping data out onto the line as fast as it can. The body of the loop consists of three actions: go fetch a packet from the (always obliging) network layer, construct an outbound frame using the variables, and send the frame on its way. Only the info field of the frame is used by this protocol, because the other fields have to do with error and flow control and there are no errors or flow control restrictions here.

The **receiver** is equally simple. Initially, it waits for something to happen, the only possibility being the arrival of an undamaged frame. Eventually, the frame arrives and the procedure wait_for_event returns, with event set to frame_arrival (which is ignored anyway). The call to from_physical_layer removes the newly arrived frame from the hardware buffer and puts it in the variable r, where the receiver code can get at it. Finally, the data portion is passed on to the network layer, and the data link layer settles back to wait for the next frame, effectively suspending itself until the frame arrives.

/* **Protocol 1 (Utopia)** provides for data transmission in one direction only, from sender to receiver. The communication channel is assumed to be error free and the receiver is assumed to be able to process all the input infinitely quickly. Consequently, the sender just sits in a loop pumping data out onto the line as fast as it can. */

```
typedef enum {frame_arrival} event_type;
#include "protocol.h"

void sender1(void)
{
        frame s;                                /* buffer for an outbound frame */
        packet buffer;                          /* buffer for an outbound packet */
        while (true)
        {
                from_ network_layer(&buffer);   /* go get something to send */
                s.info = buffer;                /* copy it into s for transmission */
                to_physical_layer(&s);          /* send it on its way */
        }                                       /* Tomorrow, and tomorrow, and tomorrow,
                                                   Creeps in this petty pace from day to day
                                                To the last syllable of recorded time. – Macbeth, V, v */
}
```

```
void receiver1(void)
{
        frame r;
        event_type_event;                       /* filled in by wait, but not used here */
        while (true)
        {
                wait_for_event(&event);         /* only possibility is frame arrival */
                from_physical_layer(&r);        /* go get the inbound frame */
                to_network_layer(&r.info);      /* pass the data to the network layer */
        }
}
```

**A utopian simplex protocol**

The utopia protocol is unrealistic because it does not handle either flow control or error correction. Its processing is close to that of an unacknowledged connectionless service that relies on higher layers to solve these problems, though even an unacknowledged connectionless service would do some error detection.

**5. Explain in detail about Stop and Wait protocol? (11 MARKS) (MAY 2017)**

**A Simplex Stop-and-Wait Protocol for an Error-Free Channel**

One solution is to build the receiver to be powerful enough to process a continuous stream of back-to-back frames. It must have sufficient buffering and processing abilities to run at the line rate and must be able to pass the frames that are received to the network layer quickly enough. However, this is a worst-case solution.

It requires dedicated hardware and can be wasteful of resources if the utilization of the link is mostly low. Moreover, it just shifts the problem of dealing with a sender that is too fast elsewhere; in this case to the network layer.

A more general solution to this problem is to have the receiver provide feedback to the sender. After having passed a packet to its network layer, the receiver sends a little dummy frame back to the sender which, in effect, gives the sender permission to transmit the next frame. After having sent a frame, the sender is required by the protocol to bide its time until the little dummy (i.e., acknowledgement) frame arrives. This delay is a simple example of a **flow control protocol.**

Protocols in which the sender sends one frame and then waits for an acknowledgement before proceeding are called **stop-and-wait**. Figure gives an example of a simplex stop-and-wait protocol.

/* **Protocol 2 (Stop-and-wait)** also provides for a one-directional flow of data from sender to receiver. The communication channel is once again assumed to be error free, as in protocol 1. However, this time the receiver has only a finite buffer capacity and a finite processing speed, so the protocol must explicitly prevent the sender from flooding the receiver with data faster than it can be handled. */

```
typedef enum {frame_arrival} event_type;
#include "protocol.h"
void sender2(void)
{
        frame s;                                /* buffer for an outbound frame */
        packet buffer;                          /* buffer for an outbound packet */
        event_type event;                       /* frame arrival is the only possibility */
        while (true)
        {
                from_network_layer(&buffer);    /* go get something to send */
                s.info = buffer;                /* copy it into s for transmission */
                to_physical_layer(&s);          /* bye-bye little frame */
                wait_for_event(&event);         /* do not proceed until given the go ahead */
        }
}

void receiver2(void)
{
        frame r, s;                             /* buffers for frames */
        event_type event;                       /* frame arrival is the only possibility */
        while (true)
        {
                wait_for_event(&event);         /* only possibility is frame arrival */
                from_physical_layer(&r);        /* go get the inbound frame */
                to_network_layer(&r.info);      /* pass the data to the network layer */
                to_physical_layer(&s);          /* send a dummy frame to awaken sender */
        }
}
```

**A simplex stop-and-wait protocol**

- In this example is simplex, going only from the sender to the receiver, frames do travel in both directions? Consequently, the communication channel between the two data link layers needs to be capable of bidirectional information transfer. However, this protocol entails a strict alternation of flow: first the sender sends a frame, then the receiver sends a frame, then the sender sends another frame, then the receiver sends another one, and so on. A half-duplex physical channel would suffice here.
- As in protocol 1, the sender starts out by fetching a packet from the network layer, using it to construct a frame, and sending it on its way. But now, unlike in protocol 1, the sender must wait until an acknowledgement frame arrives before looping back and fetching the next packet from the network layer. The sending data link layer need not even inspect the incoming frame as there is only one possibility. The incoming frame is always an acknowledgement.
- The only difference between receiver1 and receiver2 is that after delivering a packet to the network layer, receiver2 sends an acknowledgement frame back to the sender before entering the wait loop again. Because only the arrival of the frame back at the sender is important, not its contents, the receiver need not put any particular information in it.

**A Simplex Stop-and-Wait Protocol for a Noisy Channel**

Consider the normal situation of a communication channel that makes errors. Frames may be either damaged or lost completely. However, we assume that if a frame is damaged in transit, the receiver hardware will detect this when it computes the checksum.

If the frame is damaged in such a way that the checksum is nevertheless correct—an unlikely occurrence - this protocol (and all other protocols) can fail (i.e., deliver an incorrect packet to the network layer).

- The data link layer is to provide error-free, transparent communication between network layer processes.
- The network layer on machine A gives a series of packets to its data link layer, which must ensure that an identical series of packets is delivered to the network layer on machine B by its data link layer.
- In particular, the network layer on B has no way of knowing that a packet has been lost or duplicated, so the data link layer must guarantee that no combination of transmission errors, however unlikely, can cause a duplicate packet to be delivered to a network layer.

Consider the following scenario:

1. The network layer on A gives packet 1 to its data link layer. The packet is correctly received at B and passed to the network layer on B. B sends an acknowledgement frame back to A.
2. The acknowledgement frame gets lost completely. It just never arrives at all. Life would be a great deal simpler if the channel mangled and lost only data frames and not control frames, but sad to say, the channel is not very discriminating.
3. The data link layer on A eventually times out. Not having received an acknowledgement, it (incorrectly) assumes that its data frame was lost or damaged and sends the frame containing packet 1 again.

4. The duplicate frame also arrives intact at the data link layer on B and is unwittingly passed to the network layer there. If A is sending a file to B, part of the file will be duplicated (i.e., the copy of the file made by B will be incorrect and the error will not have been detected). In other words, the protocol will fail.

Clearly, what is needed is some way for the receiver to be able to distinguish a frame that it is seeing for the first time from a retransmission. The obvious way to achieve this is to have the sender put a sequence number in the header of each frame it sends. Then the receiver can check the sequence number of each arriving frame to see if it is a new frame or a duplicate to be discarded.

The only ambiguity in this protocol is between a frame, m, and its direct successor, m + 1. If frame m is lost or damaged, the receiver will not acknowledge it, so the sender will keep trying to send it. Once it has been correctly received, the receiver will send an acknowledgement to the sender. Depending upon whether the acknowledgement frame gets back to the sender correctly or not, the sender may try to send m or m + 1.

At the sender, the event that triggers the transmission of frame m + 1 is the arrival of an acknowledgement for frame m. But this situation implies that m – 1 has been correctly received, and furthermore that its acknowledgement has also been correctly received by the sender. Otherwise, the sender would not have begun with m, let alone have been considering m + 1. As a consequence, the only ambiguity is between a frame and its immediate predecessor or successor, not between the predecessor and successor themselves.

A 1-bit sequence number (0 or 1) is therefore sufficient. At each instant of time, the receiver expects a particular sequence number next. When a frame containing the correct sequence number arrives, it is accepted and passed to the network layer, then acknowledged. Then the expected sequence number is incremented modulo 2 (i.e., 0 becomes 1 and 1 becomes 0). Any arriving frame containing the wrong sequence number is rejected as a duplicate. However, the last valid acknowledgement is repeated so that the sender can eventually discover that the frame has been received.

Protocols in which the sender waits for a positive acknowledgement before advancing to the next data item are often called **ARQ (Automatic Repeat reQuest)** or **PAR (Positive Acknowledgement with Retransmission).** Like protocol 2, this one also transmits data only in one direction.

```
/* Protocol 3 (PAR) allows unidirectional data flow over an unreliable channel. */
#define MAX_SEQ 1                                   /* must be 1 for protocol 3 */
typedef enum
{
    frame_arrival, cksum err, timeout
} event_type;
```

```
#include "protocol.h"
void sender3(void)
{
        seq_nr next_frame_to_send;                  /* seq number of next outgoing frame */
        frame s;                                    /* scratch variable */
        packet buffer;                              /* buffer for an outbound packet */
        event_type event;
        next_frame_to_send = 0;                     /* initialize outbound sequence numbers */
        from_network_layer(&buffer);                /* fetch first packet */
   while (true)
  {
        s.info = buffer;                            /* construct a frame for transmission */
        s.seq = next_frame_to_send;                 /* insert sequence number in frame */
        to_physical_layer(&s);                      /* send it on its way */
        start_timer(s.seq);                         /* if answer takes too long, time out */
        wait_for_event(&event);                     /* frame_arrival, cksum_err, timeout */
        if (event == frame_arrival)
        {
                from_physical_layer(&s);            /* get the acknowledgement */
                if (s.ack == next_frame_to_send)
                {
                        stop_timer(s.ack);          /* turn the timer off */
                        from_network_layer(&buffer);/* get the next one to send */
                        inc(next_frame_to_send);    /* invert next_frame_to_send */
                }
        }
   }
}

void receiver3(void)
{
        seq_nr frame_expected;
        frame r, s;
        event_type event;
        frame_expected = 0;
  while (true)
  {
        wait_for_event(&event);                     /* possibilities: frame_arrival, cksum_err */
```

```
        if (event == frame_arrival)                    /* a valid frame has arrived */
        {
                from_physical_layer(&r);               /* go get the newly arrived frame */
                if (r.seq == frame_expected)           { /* this is what we have been waiting for */
                to_network_layer(&r.info);             /* pass the data to the network layer */
                inc(frame_expected);                   /* next time expect the other sequence nr */
        }
        s.ack = 1 – frame_expected;                    /* tell which frame is being acked */
        to_physical_layer(&s);                         /* send acknowledgement */
  }
}
}
```

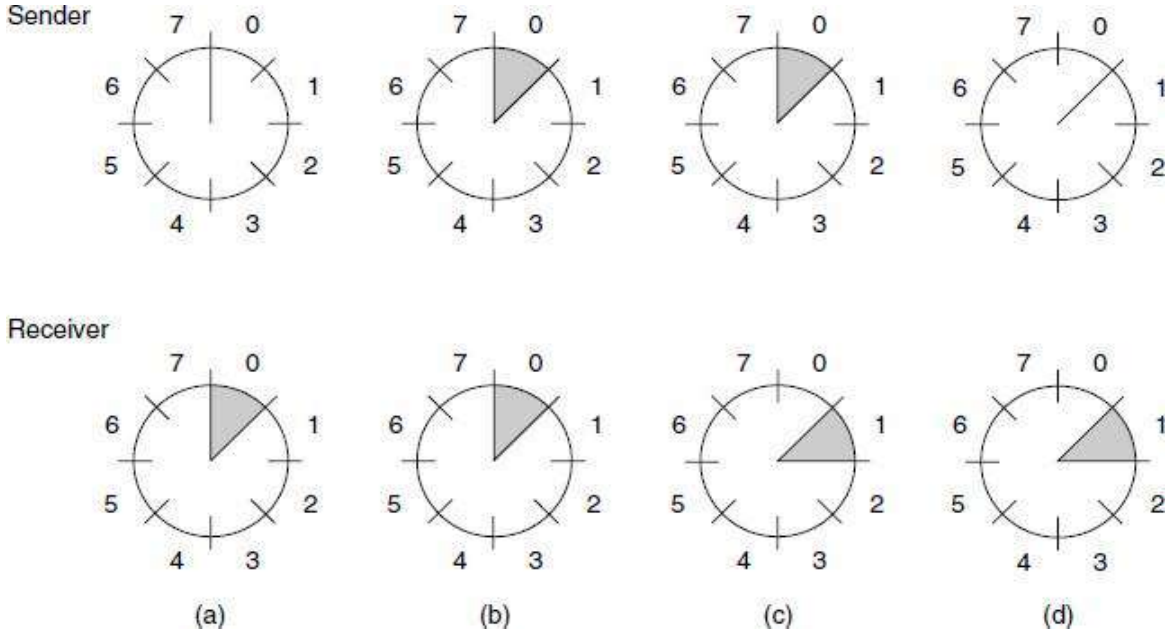
**A positive acknowledgement with retransmission protocol**

**6. Explain in detail about Sliding Window Protocols? (11 MARKS) (NOV 2016)**

The three protocols are bidirectional protocols that belong to a class called sliding window protocols.

1. One-Bit Sliding Window Protocol
2. A Protocol Using Go-Back-N
3. A Protocol Using Selective Repeat

- The stop-and-wait sliding window protocol uses n = 1, restricting the sequence numbers to 0 and 1, but more sophisticated versions can use an arbitrary n.




**A sliding window of size 1, with a 3-bit sequence number. (a) Initially. (b) After the first frame has been sent. (c) After the first frame has been received. (d) After the first acknowledgement has been received.**

- The essence of all sliding window protocols is that at any instant of time, the sender maintains a set of sequence numbers corresponding to frames it is permitted to send. These frames are said to fall within the **sending window.**
- Similarly, the receiver also maintains a **receiving window** corresponding to the set of frames it is permitted to accept. The sender's window and the receiver's window need not have the same lower and upper limits or even have the same size. In some protocols they are fixed in size, but in others they can grow or shrink over the course of time as frames are sent and received.
- Figure shows an example with a maximum window size of 1. Initially, no frames are outstanding, so the lower and upper edges of the sender's window are equal, but as time goes on, the situation progresses as shown. Unlike the sender's window, the receiver's window always remains at its initial size, rotating as the next frame is accepted and delivered to the network layer.

## 1. A One-Bit Sliding Window Protocol

Let us examine a sliding window protocol with a window size of 1. Such a protocol uses stop-and-wait since the sender transmits a frame and waits for its acknowledgement before sending the next one. Next_frame_to_send tells which frame the sender is trying to send. Similarly, frame_expected tells which frame the receiver is expecting. In both cases, 0 and 1 are the only possibilities.

```
/* Protocol 4 (Sliding window) is bidirectional. */
#define MAX_SEQ 1                                   /* must be 1 for protocol 4 */
typedef enum {frame_arrival, cksum_err, timeout} event_type;
#include "protocol.h"
void protocol4 (void)
{
    seq_nr next_frame_to_send;                      /* 0 or 1 only */
    seq_nr frame_expected;                          /* 0 or 1 only */
    frame r, s;                                     /* scratch variables */
    packet buffer;                                  /* current packet being sent */
    event_type event;
    next_frame_to_send = 0;                         /* next frame on the outbound stream */
    frame_expected = 0;                             /* frame expected next */
    from_network_layer(&buffer);                    /* fetch a packet from the network layer */
    s.info = buffer;                                /* prepare to send the initial frame */
    s.seq = next_frame_to_send;                     /* insert sequence number into frame */
    s.ack = 1-frame_expected;                       /* piggybacked ack */
    to_physical_layer(&s);                          /* transmit the frame */
    start_timer(s.seq);                             /* start the timer running */
    while (true)
    {
```

```
            wait_for_event(&event);                         /* frame arrival, cksum err, or timeout */
            if (event == frame_arrival)                     /* a frame has arrived undamaged */
            {
                from_physical_layer(&r);                    /* go get it */
                if (r.seq == frame_expected)                /* handle inbound frame stream */
                {
                    to_network_layer(&r.info);              /* pass packet to network layer */
                    inc(frame_expected);                    /* invert seq number expected next */
                }
                if (r.ack == next frame to send)            /* handle outbound frame stream */
                {
                    stop_timer(r.ack);                      /* turn the timer off */
                    from_network_layer(&buffer);            /* fetch new pkt from network layer */
                    inc(next_frame_to_send);                /* invert sender's sequence number */
                }
            }
        s.info = buffer;                                    /* construct outbound frame */
        s.seq = next_frame_to_send;                         /* insert sequence number into it */
        s.ack = 1 – frame_expected;                         /* seq number of last received frame */
        to_physical_layer(&s);                              /* transmit a frame */
        start_timer(s.seq);                                 /* start the timer running */
    }
}
```

**A 1-bit sliding window protocol**

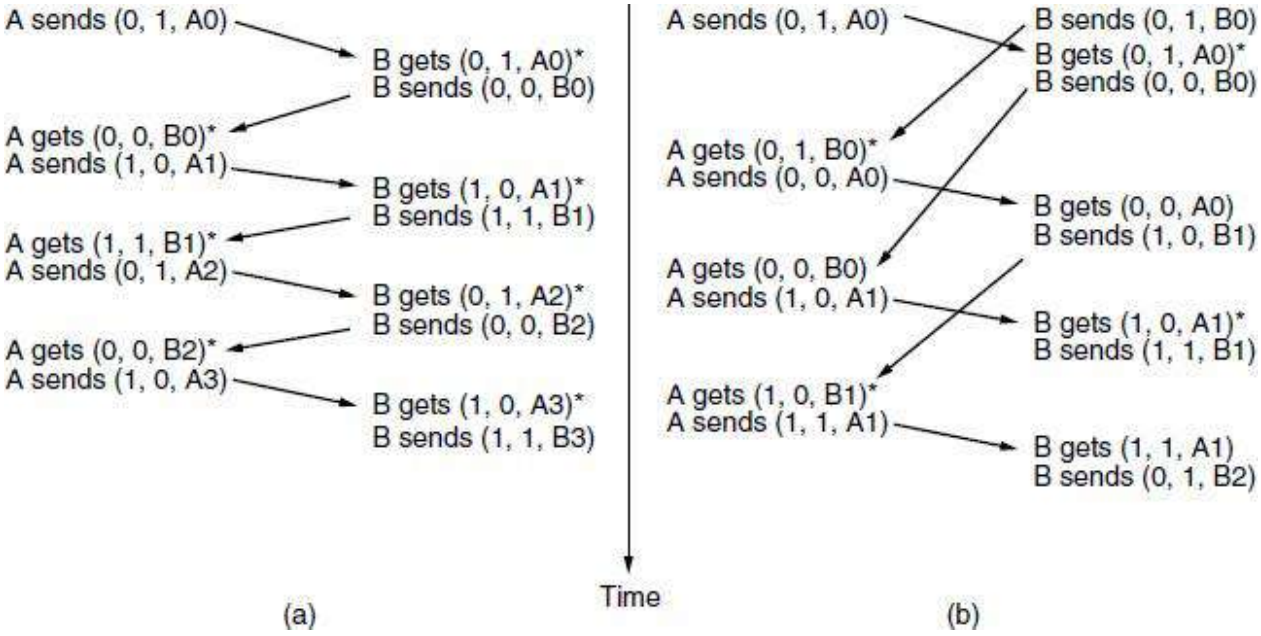
Assume that computer A is trying to send its frame 0 to computer B and that B is trying to send its frame 0 to A. Suppose that A sends a frame to B, but A's timeout interval is a little too short. Consequently, A may time out repeatedly, sending a series of identical frames, all with seq = 0 and ack = 1.

When the first valid frame arrives at computer B, it will be accepted and frame expected will be set to a value of 1. All the subsequent frames received will be rejected because B is now expecting frames with sequence number 1, not 0. Furthermore, since all the duplicates will have ack = 1 and B is still waiting for an acknowledgement of 0, B will not go and fetch a new packet from its network layer.

After every rejected duplicate comes in, B will send A a frame containing seq = 0 and ack = 0. Eventually, one of these will arrive correctly at A, causing A to begin sending the next packet. No combination of lost frames or premature timeouts can cause the protocol to deliver duplicate packets to either network layer, to skip a packet, or to deadlock. The protocol is correct.

In part (a), the normal operation of the protocol is shown. In (b) the peculiarity is illustrated. If B waits for A's first frame before sending one of its own, the sequence is as shown in (a), and every frame is accepted.

However, if A and B simultaneously initiate communication, their first frames cross, and the data link layers then get into situation (b). In (a) each frame arrival brings a new packet for the network layer; there are no duplicates. In (b) half of the frames contain duplicates, even though there are no transmission errors. Similar situations can occur as a result of premature timeouts, even when one side clearly starts first. In fact, if multiple premature timeouts occur, frames may be sent three or more times, wasting valuable bandwidth.



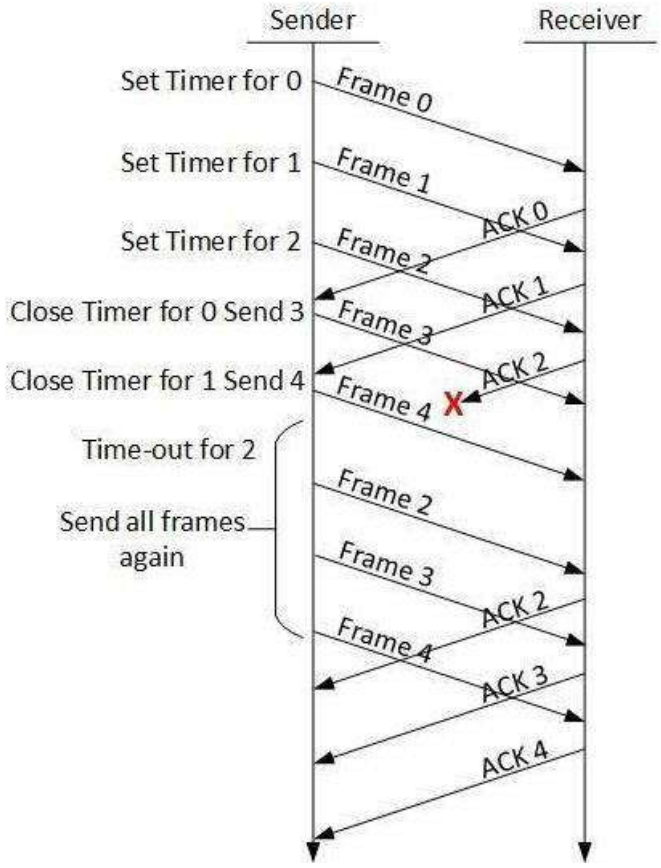

**Two scenarios for protocol 4. (a) Normal case. (b) Abnormal case. The notation is (seq, ack, packet number). An asterisk indicates where a network layer accepts a packet.**

## 2. A Protocol Using Go-Back-N

Stop and wait ARQ mechanism does not utilize the resources at their best. When the acknowledgement is received, the sender sits idle and does nothing. In Go-Back-N ARQ method, both sender and receiver maintain a window.

The sending-window size enables the sender to send multiple frames without receiving the acknowledgement of the previous ones. The receiving-window enables the receiver to receive multiple frames and acknowledge them. The receiver keeps track of incoming frame's sequence number.

When the sender sends all the frames in window, it checks up to what sequence number it has received positive acknowledgement. If all frames are positively acknowledged, the sender sends next set of frames. If sender finds that it has received NACK or has not receive any ACK for a particular frame, it retransmits all the frames after which it does not receive any positive ACK.

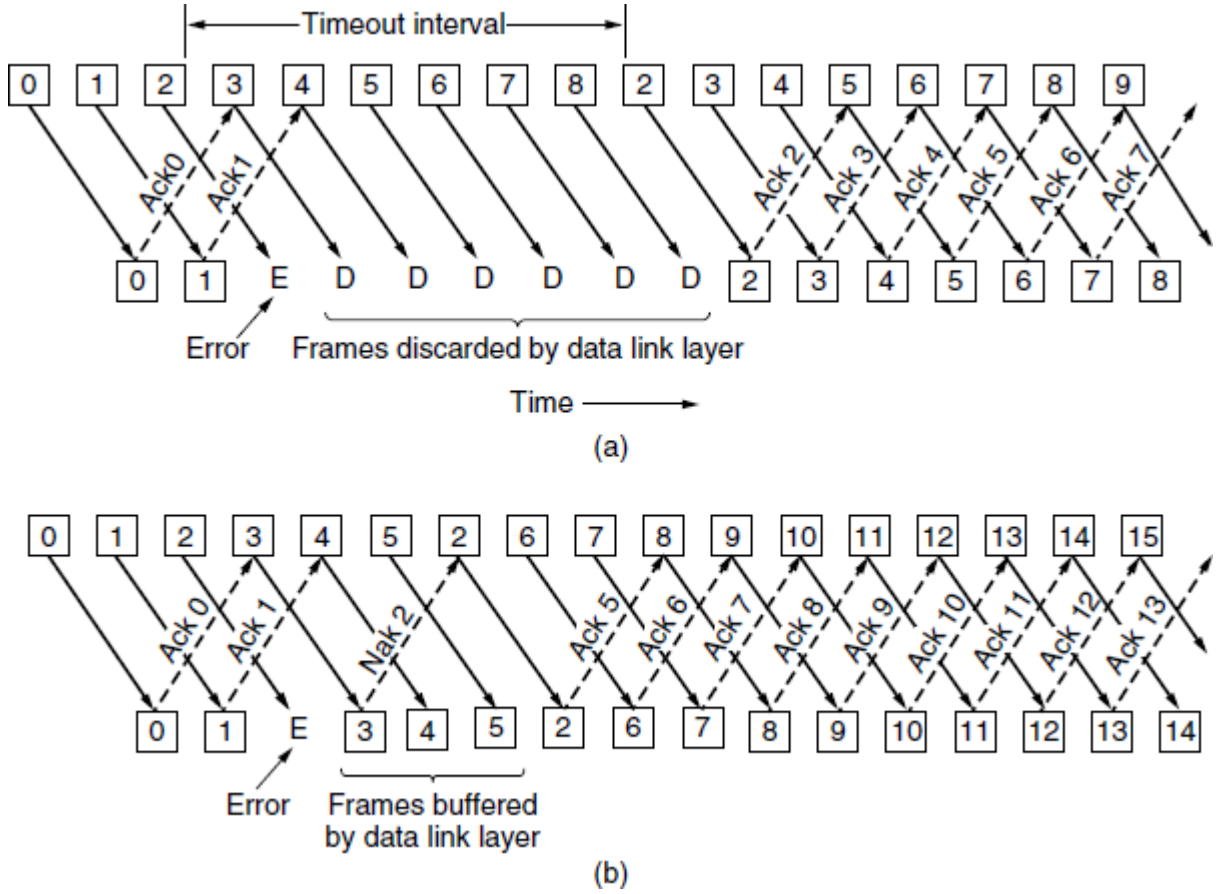



In **go-back-n**, is for the receiver simply to discard all subsequent frames, sending no acknowledgements for the discarded frames. This strategy corresponds to a receive window of size 1. In other words, the data link layer refuses to accept any frame except the next one it must give to the network layer.

If the sender's window fills up before the timer runs out, the pipeline will begin to empty. Eventually, the sender will time out and retransmit all unacknowledged frames in order, starting with the damaged or lost one. This approach can waste a lot of bandwidth if the error rate is high.

In Figure (b) we see go-back-n for the case in which the receiver's window is large. Frames 0 and 1 are correctly received and acknowledged. Frame 2, however, is damaged or lost. The sender, unaware of this problem, continues to send frames until the timer for frame 2 expires. Then it backs up to frame 2 and starts over with it, sending 2, 3, 4, etc. all over again.

The other general strategy for handling errors when frames are pipelined is called **selective repeat.** When it is used, a bad frame that is received is discarded, but any good frames received after it are accepted and buffered. When the sender times out, only the oldest unacknowledged frame is retransmitted. If that frame arrives correctly, the receiver can deliver to the network layer, in sequence, all the frames it has buffered. Selective repeat corresponds to a receiver window larger than 1. This approach can require large amounts of data link layer memory if the window is large.




**Pipelining and error recovery. Effect of an error when (a) receiver's window size is 1 and (b) receiver's window size is large**

Selective repeat is often combined with having the receiver send a **negative acknowledgement (NAK)** when it detects an error, for example, when it receives a checksum error or a frame out of sequence. NAKs stimulate retransmission before the corresponding timer expires and thus improve performance.

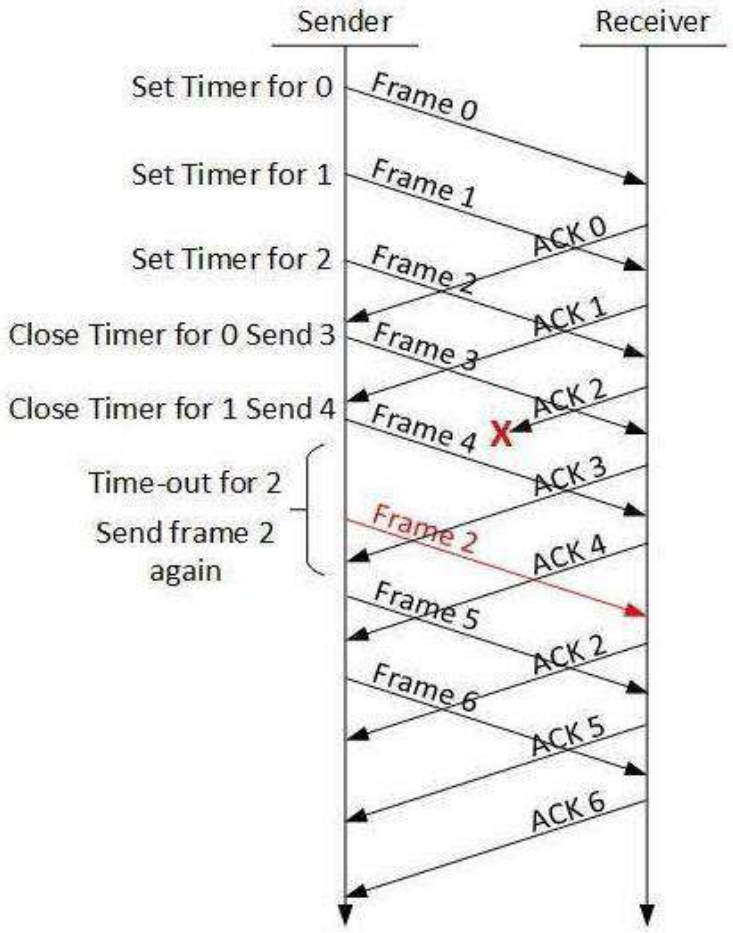
In Figure (b), frames 0 and 1 are again correctly received and acknowledged and frame 2 is lost. When frame 3 arrives at the receiver, the data link layer there notices that it has missed a frame, so it sends back a NAK for 2 but buffers 3.

When frames 4 and 5 arrive, they, too, are buffered by the data link layer instead of being passed to the network layer. Eventually, the NAK 2 gets back to the sender, which immediately resends frame 2. When that arrives, the data link layer now has 2, 3, 4, and 5 and can pass all of them to the network layer in the correct order. It can also acknowledge all frames up to and including 5, as shown in the figure. If the NAK should get lost, eventually the sender will time out for frame 2 and send it (and only it) of its own accord, but that may be a quite a while later.

When an acknowledgement comes in for frame n, frames n − 1, n − 2, and so on are also automatically acknowledged. This type of acknowledgement is called a **cumulative acknowledgement.**

## 3. A Protocol Using Selective Repeat

In Go-back-N ARQ, it is assumed that the receiver does not have any buffer space for its window size and has to process each frame as it comes. This enforces the sender to retransmit all the frames which are not acknowledged.




In **Selective-Repeat Automatic Repeat Request (ARQ)**, the receiver while keeping track of sequence numbers, buffers the frames in memory and sends NACK for only frame which is missing or damaged. The sender in this case, sends only packet for which NACK is received.

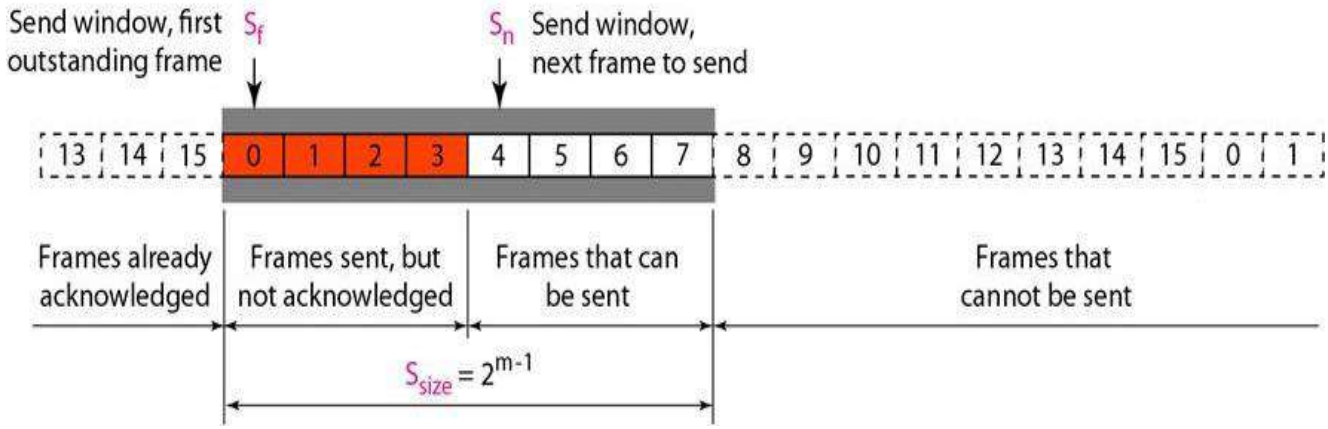
- Go-Back-N ARQ simplifies the process at the receiver site. The receiver keeps track of only one variable, and there is no need to buffer out-of-order frames; they are simply discarded. However, this protocol is very inefficient for a noisy link.
- In a noisy link a frame has a higher probability of damage, which means the resending of multiple frames. This resending uses up the bandwidth and slows down the transmission.
- For noisy links, there is another mechanism that does not resend N frames when just one frame is damaged; only the damaged frame is resent. This mechanism is called **Selective Repeat ARQ**.
- It is more efficient for noisy links, but the processing at the receiver is more complex.

### Windows

- The Selective Repeat Protocol also uses two windows: a send window and a receive window. However, there are differences between the windows in this protocol and the ones in Go-Back-N.

1. First, the size of the send window is much smaller; it is $2^{m-1}$.
2. Second, the receive window is the same size as the send window.

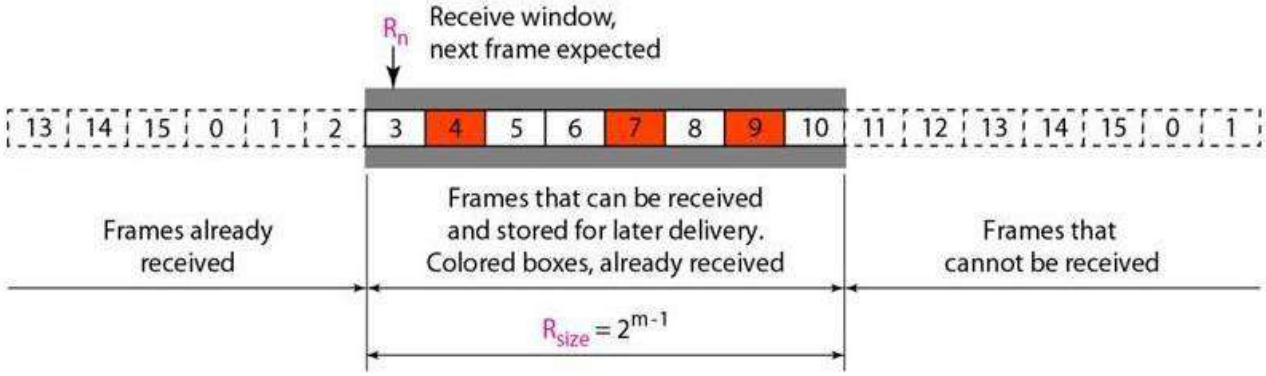
- The send window maximum size can be $2^{m-1}$. For example, if m = 4, the sequence numbers go from 0 to 15, but the size of the window is just 8 (it is 15 in the Go-Back-N Protocol). The smaller window size means less efficiency in filling the pipe, but the fact that there are fewer duplicate frames can compensate for this.




**Send window for Selective Repeat ARQ**

The receive window in Selective Repeat is totally different from the one in Go-Back-N. First, the size of the receive window is the same as the size of the send window ($2^{m-1}$). The Selective Repeat Protocol allows as many frames as the size of the receive window to arrive out of order and be kept until there is a set of in-order frames to be delivered to the network layer.

Because the sizes of the send window and receive window are the same, all the frames in the send frame can arrive out of order and be stored until they can be delivered. However, to mention that the receiver never delivers packets out of order to the network layer.




**Receive window for Selective Repeat ARQ**

**7. Explain in detail about Medium Access Control Sub-layer? (11 MARKS)**

- A broadcast channels are sometimes referred to as **multi-access channels** or **random access channels.**
- The protocols used to determine who goes next on a multi-access channel belong to a sub-layer of the data link layer called the **MAC (Medium Access Control) sublayer**.
- The MAC sublayer is especially important in LANs, particularly wireless ones because wireless is naturally a broadcast channel.
- WANs, in contrast, use point-to-point links, except for satellite networks.

**The Channel Allocation Problem:**

- The central theme is how to allocate a single broadcast channel among competing users.
- The channel might be a portion of the wireless spectrum in a geographic region, or a single wire or optical fiber to which multiple nodes are connected.
- In both cases, the channel connects each user to all other users and any user who makes full use of the channel interferes with other users who also wish to use the channel.

The two types of Channel Allocation Problems are

1. Static Channel Allocation
2. Dynamic Channel Allocation

**Static Channel Allocation**

- The traditional way of allocating a single channel, such as a telephone trunk, among multiple competing users is to chop up its capacity by using one of the multiplexing schemes such as **FDM (Frequency Division Multiplexing).**
- If there are N users, the bandwidth is divided into N equal-sized portions, with each user being assigned one portion.
- Since each user has a private frequency band, there is now no interference among users.
- When there is only a small and constant number of users, each of which has a steady stream or a heavy load of traffic, this division is a simple and efficient allocation mechanism.
- A wireless example is FM radio stations.
- Each station gets a portion of the FM band and uses it most of the time to broadcast its signal.

However, when the number of senders is large and varying or the traffic is bursty, FDM presents some problems. If the spectrum is cut up into N regions and fewer than N users are currently interested in communicating, a large piece of valuable spectrum will be wasted. And if more than N users want to communicate, some of them will be denied permission for lack of bandwidth, even if some of the users who have been assigned a frequency band hardly ever transmit or receive anything.

A static allocation is a poor fit to most computer systems, in which data traffic is extremely bursty, often with peak traffic to mean traffic ratios of 1000:1.

The poor performance of static FDM can easily be seen with a simple queuing theory calculation.

Let us start by finding the mean time delay, T, to send a frame onto a channel of capacity C bps. We assume that the frames arrive randomly with an average arrival rate of λ frames/sec, and that the frames vary in length with an average length of 1/μ bits. With these parameters, the service rate of the channel is μC frames/sec.

A standard queuing theory result is

$$T = \frac{1}{\mu C - \lambda}$$

In our example, if C is 100 Mbps, the mean frame length, 1/μ, is 10,000 bits, and the frame arrival rate, λ, is 5000 frames/sec, then T = 200 μsec. Note that if we ignored the queueing delay and just asked how long it takes to send a 10,000- bit frame on a 100-Mbps network, we would get the (incorrect) answer of 100 μsec. That result only holds when there is no contention for the channel.

Now let us divide the single channel into N independent subchannels, each with capacity C /N bps. The mean input rate on each of the subchannels will now be λ/N. Recomputing T, we get

$$T_N = \frac{1}{\mu(C/N) - (\lambda/N)} = \frac{N}{\mu C - \lambda} = NT$$

To use **Time Division Multiplexing (TDM)** and allocate each user every Nth time slot, if a user does not use the allocated slot. The same would hold if we split up the networks physically. Using our previous example again, if we were to replace the 100-Mbps network with 10 networks of 10 Mbps each and statically allocate each user to one of them, the mean delay would jump from 200 μsec to 2 msec.

## 2. Dynamic Channel Allocation

The following five key assumptions are

1. **Independent Traffic.**

   The model consists of N independent stations (e.g., computers, telephones), each with a program or user that generates frames for transmission. The expected number of frames generated in an interval of length Δt is λΔt, where λ is a constant (the arrival rate of new frames). Once a frame has been generated, the station is blocked and does nothing until the frame has been successfully transmitted.

2. **Single Channel**.

   A single channel is available for all communication. All stations can transmit on it and all can receive from it. The stations are assumed to be equally capable, though protocols may assign them different roles (e.g., priorities).

3. **Observable Collisions**.

If two frames are transmitted simultaneously, they overlap in time and the resulting signal is garbled. This event is called a collision. All stations can detect that a collision has occurred. A collided frame must be transmitted again later. No errors other than those generated by collisions occur.

4. **Continuous or Slotted Time**.

Time may be assumed continuous, in which case frame transmission can begin at any instant. Alternatively, time may be slotted or divided into discrete intervals (called slots). Frame transmissions must then begin at the start of a slot. A slot may contain 0, 1, or more frames, corresponding to an idle slot, a successful transmission, or a collision, respectively.

5. **Carrier Sense or No Carrier Sense**.

With the carrier sense assumption, stations can tell if the channel is in use before trying to use it. No station will attempt to use the channel while it is sensed as busy. If there is no carrier sense, stations cannot sense the channel before trying to use it. They just go ahead and transmit. Only later can they determine whether the transmission was successful.

**8. Explain the ALOHA in Multiple Access Protocol? (11 MARKS)**

- ALOHA, the earliest random access method, was developed at the University of Hawaii in early 1970. It was designed for a radio (wireless) LAN, but it can be used on any shared medium.
- There are potential collisions in this arrangement. The medium is shared between the stations. When a station sends data, another station may attempt to do so at the same time. The data from the two stations collide and become garbled.

- They found used short-range radios, with each user terminal sharing the same upstream frequency to send frames to the central computer. It included a simple and elegant method to solve the channel allocation problem. Their work has been extended by many researchers since then (Schwartz and Abramson, 2009).
- Although Abramson's work, called the ALOHA system, used ground based radio broadcasting, the basic idea is applicable to any system in which uncoordinated users are competing for the use of a single shared channel.

The two versions of ALOHA are

1. **Pure ALOHA**
2. **Slotted ALOHA**

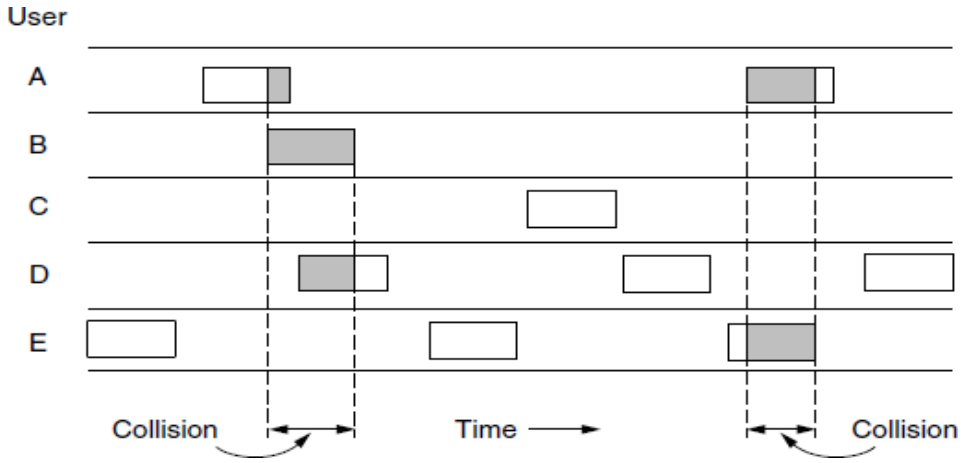
They differ with respect to whether time is continuous, as in the pure version, or divided into discrete slots into which all frames must fit.

**Pure ALOHA**

The basic idea of an ALOHA system is simple:

- Let users transmit whenever they have data to be sent. There will be collisions, of course, and the colliding frames will be damaged. Senders need some way to find out if this is the case.
- In the ALOHA system, after each station has sent its frame to the central computer, this computer rebroadcasts the frame to all of the stations.
- A sending station can thus listen for the broadcast from the hub to see if its frame has gotten through.
- In other systems, such as wired LANs, the sender might be able to listen for collisions while transmitting.
- If the frame was destroyed, the sender just waits a random amount of time and sends it again.
- The waiting time must be random or the same frames will collide over and over, in lockstep.
- Systems in which multiple users share a common channel in a way that can lead to conflicts are known as **contention systems.**

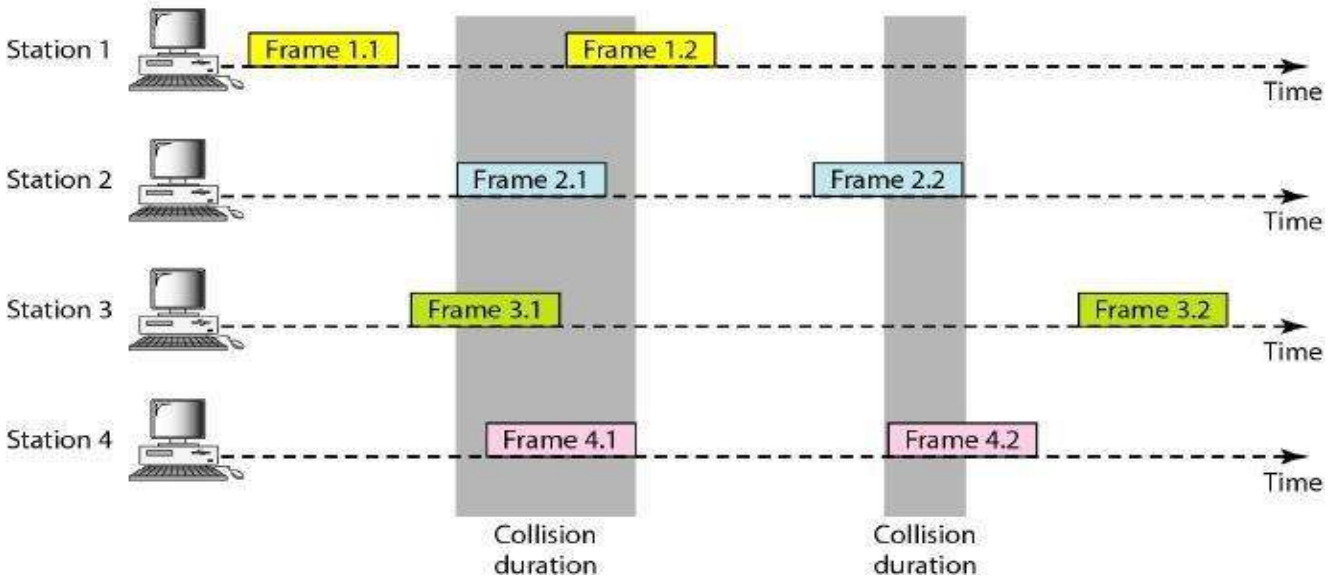
A sketch of frame generation in an ALOHA system is given in Figure.




**In pure ALOHA, frames are transmitted at completely arbitrary times**

- The frames all the same length because the throughput of ALOHA systems is maximized by having a uniform frame size rather than by allowing variable- length frames.
- Whenever two frames try to occupy the channel at the same time, there will be a collision (as seen in Figure) and both will be garbled.
- If the first bit of a new frame overlaps with just the last bit of a frame that has almost finished, both frames will be totally destroyed (i.e., have incorrect checksums) and both willhave to be retransmitted later.

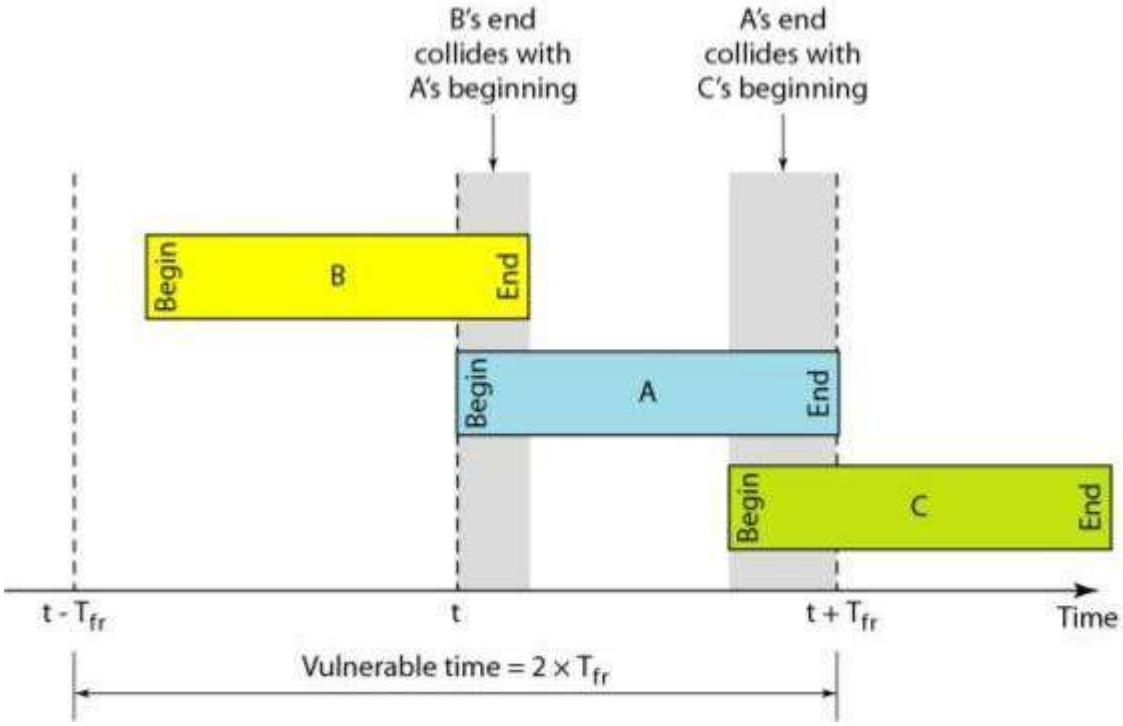
- The original ALOHA protocol is called **pure ALOHA.**
- This is a simple, but elegant protocol.
- The idea is that each station sends a frame whenever it has a frame to send.
- However, since there is only one channel to share, there is the possibility of collision between frames from different stations.




**Frames in a pure ALOHA network**

There are four stations (unrealistic assumption) that contend with one another for access to the shared channel. The figure shows that each station sends two frames; there are a total of eight frames on the shared medium. Some of these frames collide because multiple frames are in contention for the shared channel.

Figure shows that only two frames survive: frame 1.1 from station 1 and frame 3.2 from station 3. We need to mention that even if one bit of a frame coexists on the channel with one bit from another frame, there is a collision and both will be destroyed.




**Vulnerable time for pure ALOHA protocol**

It is obvious that we need to resend the frames that have been destroyed during transmission. The pure ALOHA protocol relies on acknowledgments from the receiver. When a station sends a frame, it

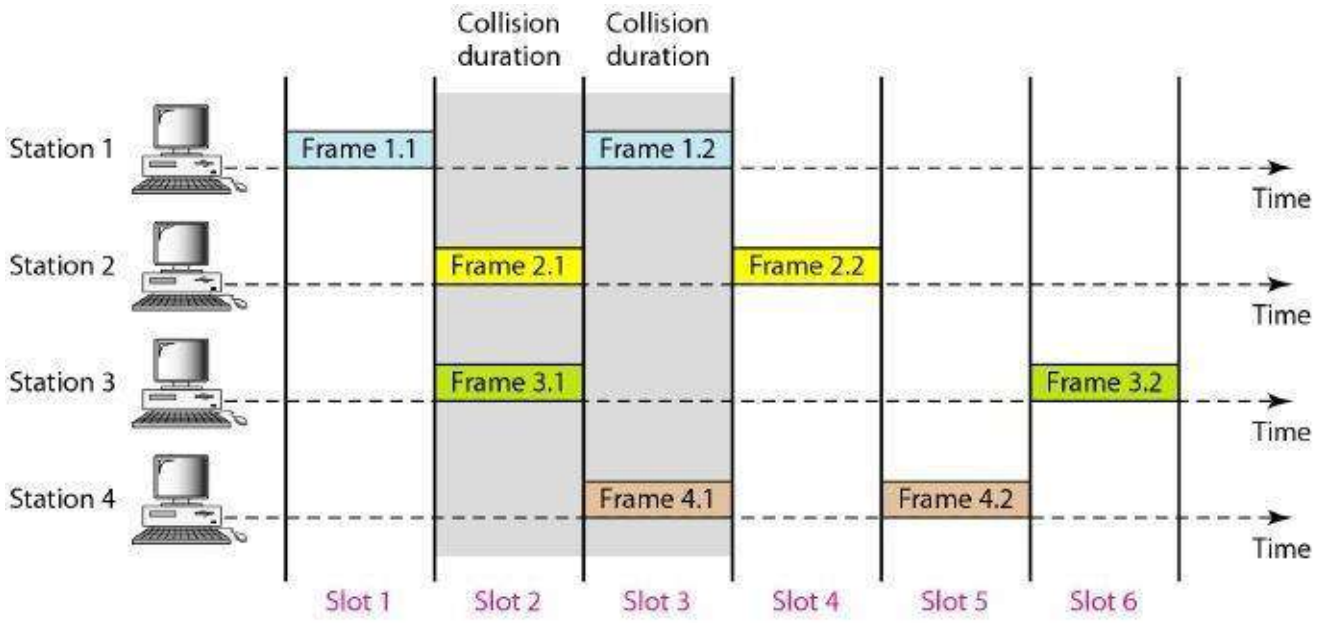
expects the receiver to send an acknowledgment. If the acknowledgment does not arrive after a time-out period, the station assumes that the frame (or the acknowledgment) has been destroyed and resends the frame.

Let us find the length of time, the **vulnerable time**, in which there is a possibility of collision. We assume that the stations send fixed-length frames with each frame taking $T_{fr}$ S to send.

**Slotted ALOHA**

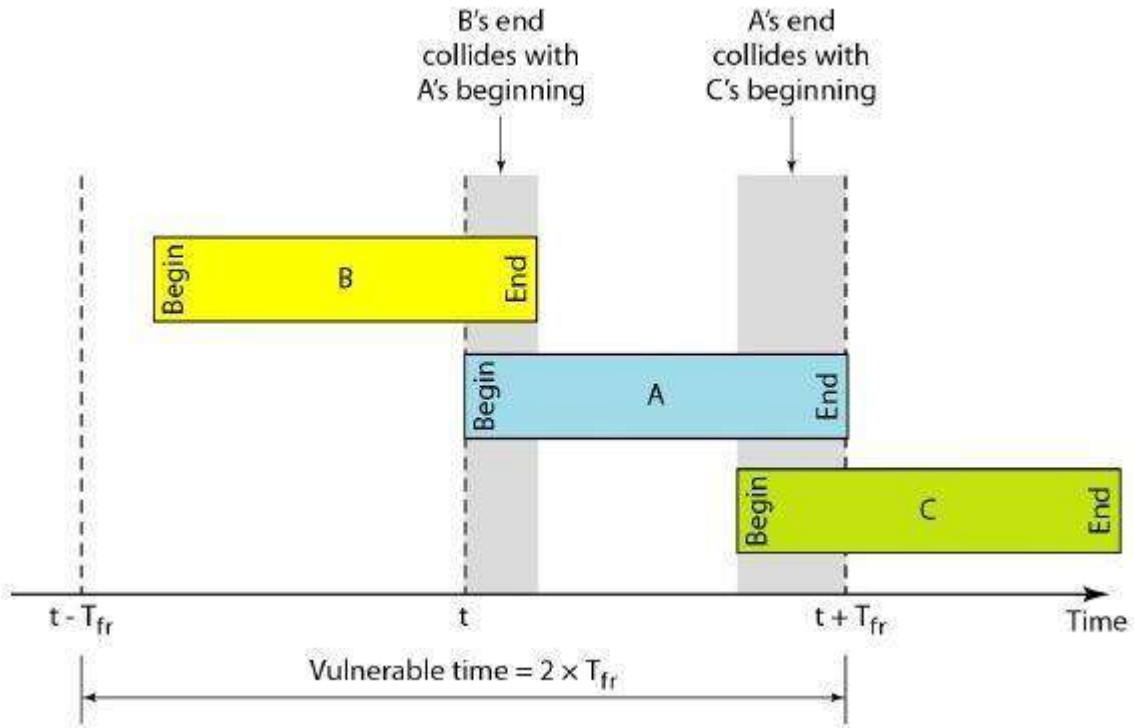
Roberts (1972) published a method for doubling the capacity of an ALOHA system. His proposal was to divide time into discrete intervals called **slots**, each interval corresponding to one frame. This approach requires the users to agree on slot boundaries. One way to achieve synchronization would be to have one special station emit a pip at the start of each interval, like a clock.

In slotted ALOHA we divide the time into slots of $T_{fr}$s and force the station to send only at the beginning of the time slot.




**Frames in a slotted ALOHA network**

- Because a station is allowed to send only at the beginning of the synchronized time slot, if a station misses this moment, it must wait until the beginning of the next time slot. This means that the station which started at the beginning of this slot has already finished sending its frame.
- Of course, there is still the possibility of collision if two stations try to send at the beginning of the same time slot. However, the vulnerable time is now reduced to one-half, equal to $T_{fr}$.




**Vulnerable time for slottedALOHA protocol**

Figure shows that the vulnerable time for slotted ALOHA is one-half that of pure ALOHA.

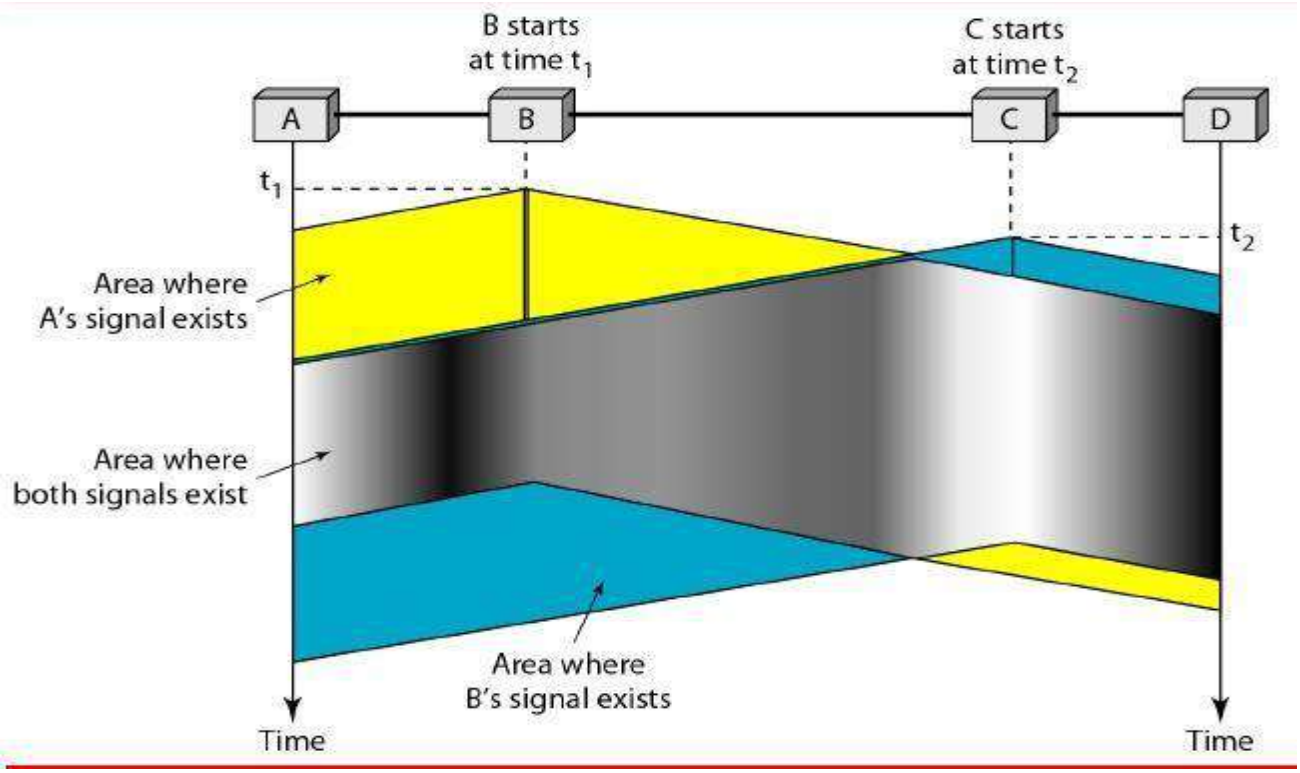
**Slotted ALOHA vulnerable time = $T_{fr}$**

**9. Explain in detail about CSMA protocols? (11 MARKS) (NOV 2015)(MAY 2016)**

- Carrier sense multiple access (CSMA) requires that each station first listen to the medium (or check the state of the medium) before sending.
- In other words, CSMA is based on the principle "sense before transmit" or "listen before talk."

CSMA can reduce the possibility of collision, but it cannot eliminate it. The reason for this is shown in Figure a space and time model of a CSMA network. Stations are connected to a shared channel (usually a dedicated medium).

- The possibility of collision still exists because of propagation delay; when a station sends a frame, it still takes time (although very short) for the first bit to reach every station and for every station to sense it.
- In other words, a station may sense the medium and find it idle, only because the first bit sent by another station has not yet been received.




**Space/time model of the collision in CSMA**

**Persistent and Non-persistent CSMA**

- The first carrier sense protocol that we will study here is called **1-persistent CSMA (Carrier Sense Multiple Access).**
- When a station has data to send, it first listens to the channel to see if anyone else is transmitting at that moment.
- If the channel is idle, the stations sends its data. Otherwise, if the channel is busy, the station just waits until it becomes idle. Then the station transmits a frame.

- If a collision occurs, the station waits a random amount of time and starts all over again.
- The protocol is called **1-persistent** because the station transmits with a probability of 1 when it finds the channel idle.

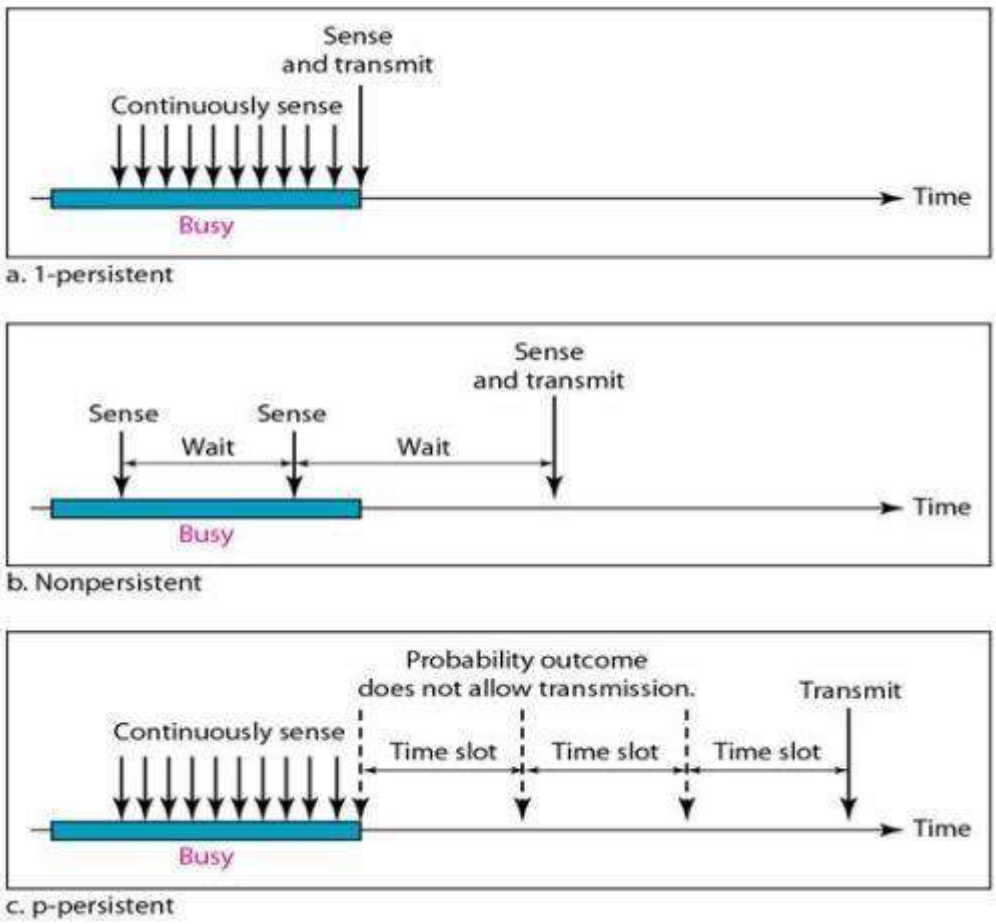
**1-Persistent**

- The 1-persistent method is simple and straightforward.
- In this method, after the station finds the line idle, it sends its frame immediately (with probability 1).
- This method has the highest chance of collision because two or more stations may find the line idle and send their frames immediately.

**Non-Persistent**

- In the non-persistent method, a station that has a frame to send senses the line.
- If the line is idle, it sends immediately.
- If the line is not idle, it waits a random amount of time and then senses the line again.
- The non-persistent approach reduces the chance of collision because it is unlikely that two or more stations will wait the same amount of time and retry to send simultaneously.

- However, this method reduces the efficiency of the network because the medium remains idle when there may be stations with frames to send.




**Behavior of three persistence methods**

**p-P ersistent**

- The p-persistent method is used if the channel has time slots with a slot duration equal to or greater than the maximum propagation time.
- The p-persistent approach combines the advantages of the other two strategies.
- It reduces the chance of collision and improves efficiency.

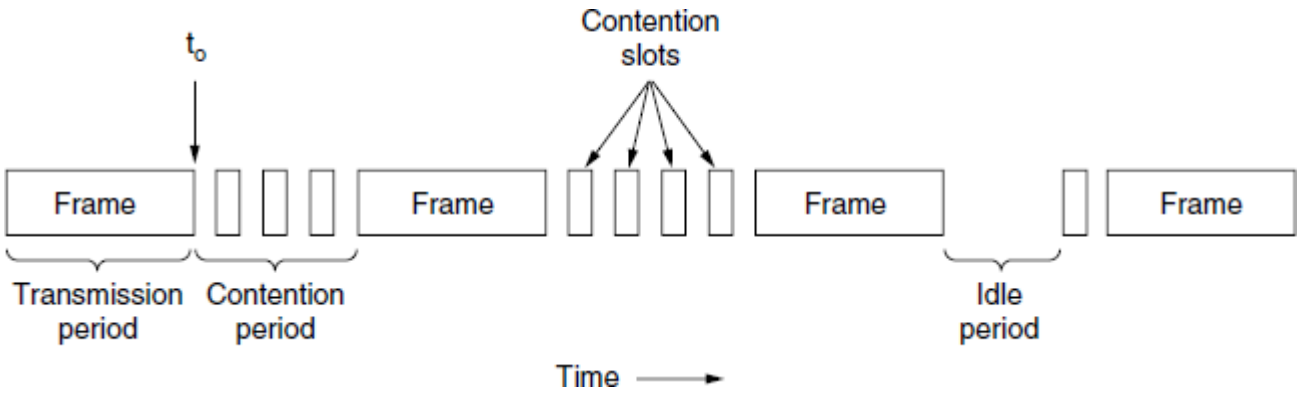
In this method, after the station finds the line idle it follows these steps:

1. With probability p, the station sends its frame.
2. With probability q = 1 - p, the station waits for the beginning of the next time slot and checks the line again.
   a) If the line is idle, it goes to step 1.
   b) If the line is busy, it acts as though a collision has occurred and uses the backoff procedure.

**Carrier Sense Multiple Access with Collision Detection (CSMA/CD)**

- Carrier sense multiple access with collision detection (CSMA/CD) augments the algorithm to handle the collision.
- In this method, a station monitors the medium after it sends a frame to see if the transmission was successful. If so, the station is finished. If, however, there is a collision, the frame is sent again.

- CSMA/CD (CSMA with Collision Detection), is the basis of the classic Ethernet LAN.
- It is important that collision detection is an analog process.
- The station's hardware must listen to the channel while it is transmitting.
- If the signal it reads back is different from the signal it is putting out, it knows that a collision is occurring.
- The implications are that a received signal must not be tiny compared to the transmitted signal (which is difficult for wireless, as received signals may be 1,000,000 times weaker than transmitted signals) and that the modulation must be chosen to allow collisions to be detected.




**CSMA/CD can be in contention, transmission, or idle state**

CSMA/CD, as well as many other LAN protocols, uses the conceptual model of Figure.

- At the point marked t 0, a station has finished transmitting its frame.
- Any other station having a frame to send may now attempt to do so.
- If two or more stations decide to transmit simultaneously, there will be a collision.
- If a station detects a collision, it aborts its transmission, waits a random period of time, and then tries again (assuming that no other station has started transmitting in the meantime).
- CSMA/CD will consist of alternating contention and transmission periods, with idle periods occurring when all stations are quiet (e.g., for lack of work).

**10. Explain the various Collision-Free Protocols? (11 MARKS)**

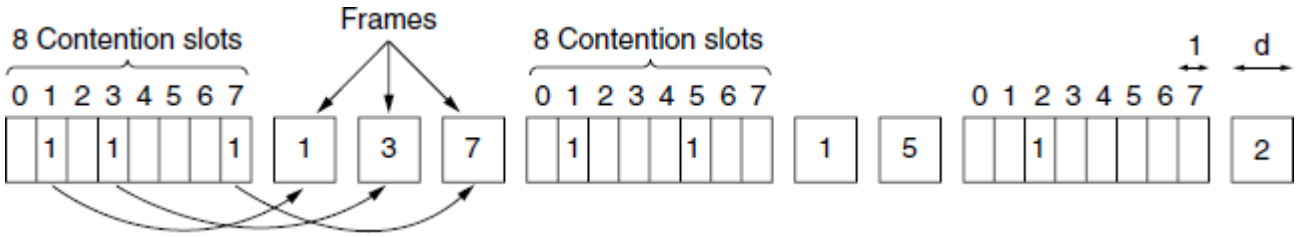
- Provides in order access to shared medium so that every station has chance to transfer (fair protocol)
- Eliminates collision completely

Three methods for controlled access:

1. A Bit-Map Protocol
2. Token Passing
3. Binary Countdown

**A Bit-Map Protocol**

- In our first collision-free protocol, the basic **bit-map method**, each contention period consists of exactly N slots.
- If station 0 has a frame to send, it transmits a 1 bit during the slot 0.
- No other station is allowed to transmit during this slot.
- Regardless of what station 0 does, station 1 gets the opportunity to transmit a 1 bit during slot 1, but only if it has a frame queued.
- In general, station j may announce that it has a frame to send by inserting a 1 bit into slot j.
- After all N slots have passed by, each station has complete knowledge of which stations wish to transmit.
- At that point, they begin transmitting frames in numerical order.




**The basic bit-map protocol**

Since everyone agrees on who goes next, there will never be any collisions. After the last ready station has transmitted its frame, an event all stations can easily monitor, another N-bit contention period is begun. If a station becomes ready just after its bit slot has passed by, it is out of luck and must remain silent until every station has had a chance and the bit map has come around again.

Protocols like this in which the desire to transmit is broadcast before the actual transmission are called **reservation protocols** because they reserve channel ownership in advance and prevent collisions.

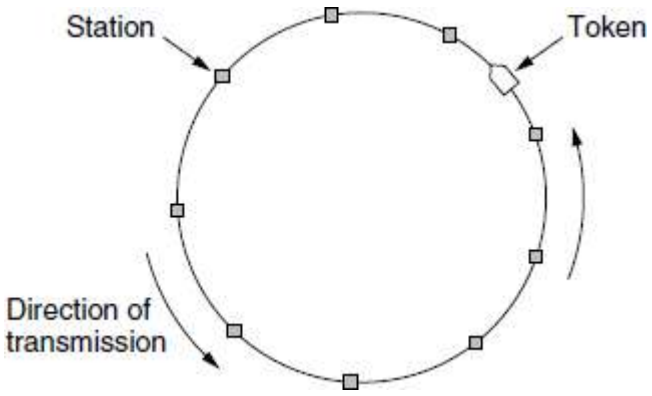
- When a station needs to send a data frame, it makes a reservation in its own mini-slot.
- By listening to the reservation interval, every station knows which stations will transfer frames, and in which order.
- The stations that made reservations can send their data frames after the reservation frame.

**Token Passing**

- The essence of the bit-map protocol is that it lets every station transmit a frame in turn in a predefined order.
- Another way to accomplish the same thing is to pass a small message called a **token** from one station to the next in the same predefined order.
- The token represents permission to send.

- If a station has a frame queued for transmission when it receives the token, it can send that frame before it passes the token to the next station.
- If it has no queued frame, it simply passes the token.

- In a token ring protocol, the topology of the network is used to define the order in which stations send.
- The stations are connected one to the next in a single ring.
- Passing the token to the next station then simply consists of receiving the token in from one direction and transmitting it out in the other direction.




**Token Ring**

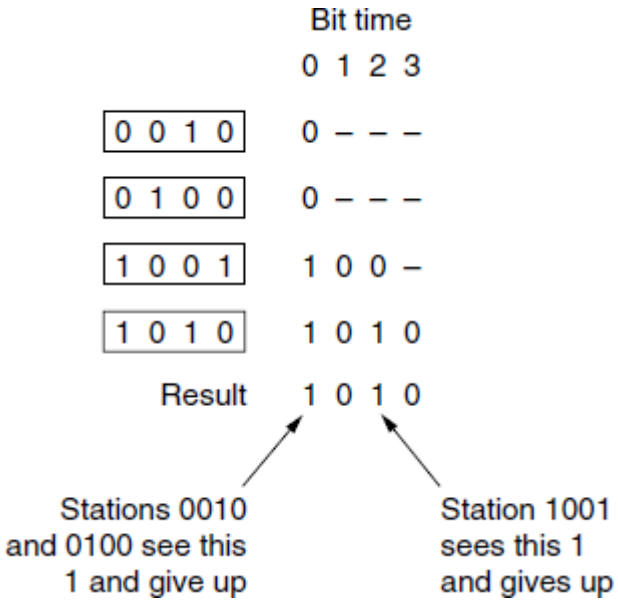
Frames are also transmitted in the direction of the token. This way they will circulate around the ring and reach whichever station is the destination. However, to stop the frame circulating indefinitely (like the token), some station needs to remove it from the ring. This station may be either the one that originally sent the frame, after it has gone through a complete cycle, or the station that was the intended recipient of the frame.

The channel connecting the stations might instead be a single long bus. Each station then uses the bus to send the token to the next station in the predefined sequence. Possession of the token allows a station to use the bus to send one frame, as before. This protocol is called **token bus.**

**Binary Countdown**

- A problem with the basic bit-map protocol, and by extension token passing, is that the overhead is 1 bit per station, so it does not scale well to networks with thousands of stations.
- By using binary station addresses with a channel that combines transmissions.
- A station wanting to use the channel now broadcasts its address as a binary bit string, starting with the high order bit.
- All addresses are assumed to be the same length.
- The bits in each address position from different stations are BOOLEAN ORed together by the channel when they are sent at the same time. This protocol **binary countdown.**

To avoid conflicts, an arbitration rule must be applied: as soon as a station sees that a high-order bit position that is 0 in its address has been overwritten with a 1, it gives up. For example, if stations 0010, 0100, 1001, and 1010 are all trying to get the channel, in the first bit time the stations transmit 0, 0, 1, and 1, respectively.




**The binary countdown protocol. A dash indicates silence**

These are ORed together to form a 1. Stations 0010 and 0100 see the 1 and know that a higher-numbered station is competing for the channel, so they give up for the current round. Stations 1001 and 1010 continue.

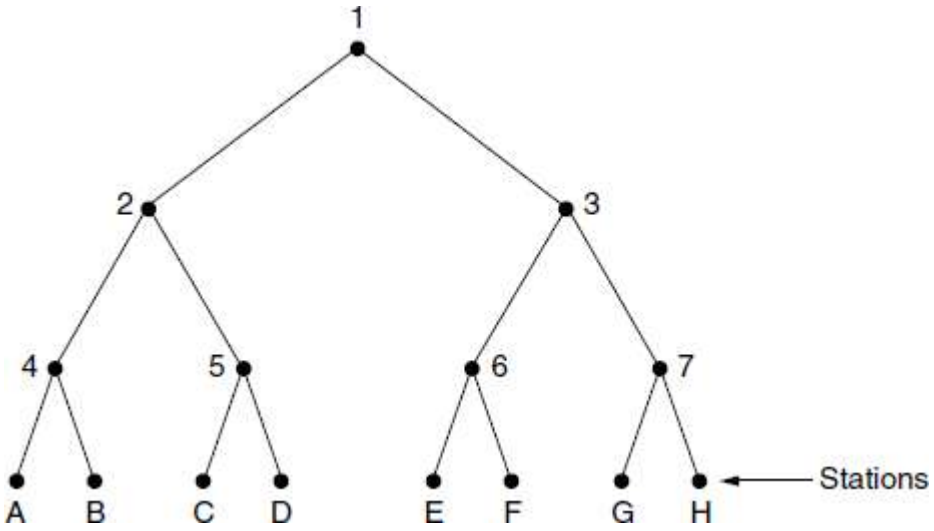
The next bit is 0, and both stations continue. The next bit is 1, so station 1001 gives up. The winner is station 1010 because it has the highest address. After winning the bidding, it may now transmit a frame, after which another bidding cycle starts. The protocol is illustrated in Figure. It has the property that higher-numbered stations have a higher priority than lower-numbered stations, which may be either good or bad, depending on the context.

**11. Explain in detail about Limited-Contention Protocols? (5 MARKS)**

The best properties of the contention and collision-free protocols, arriving at a new protocol that used contention at low load to provide low delay, but used a collision-free technique at high load to provide good channel efficiency. Such protocols is called **limited-contention protocols.**

**The Adaptive Tree Walk Protocol**

The following is the method of adaptive tree protocol. For the computerized version of this algorithm, it is convenient to think of the stations as the leaves of a binary tree, as illustrated in Figure.




**The tree for eight stations**

In the first contention slot following a successful frame transmission, slot 0, all stations are permitted to try to acquire the channel. If one of them does so, fine. If there is a collision, then during slot 1 only those stations falling under node 2 in the tree may compete. If one of them acquires the channel, the slot following the frame is reserved for those stations under node 3. If, on the other hand, two or more stations under node 2 want to transmit, there will be a collision during slot 1, in which case it is node 4's turn during slot 2.

In essence, if a collision occurs during slot 0, the entire tree is searched, depth first, to locate already stations. Each bit slot is associated with some particular node in the tree. If a collision occurs, the search continues recursively with the node's left and right children. If a bit slot is idle or if only one station transmits in it, the searching of its node can stop because all ready stations have been located.

**12. Explain in detail about Wireless LAN 802.11 Protocol architecture? (11 MARKS)**

- Wireless LANs are increasingly popular, and homes, offices, cafes, libraries, airports, zoos, and other public places are being outfitted with them to connect computers, PDAs, and smart phones to the Internet.
- Wireless LANs can also be used to let two or more nearby computers communicate without using the Internet.

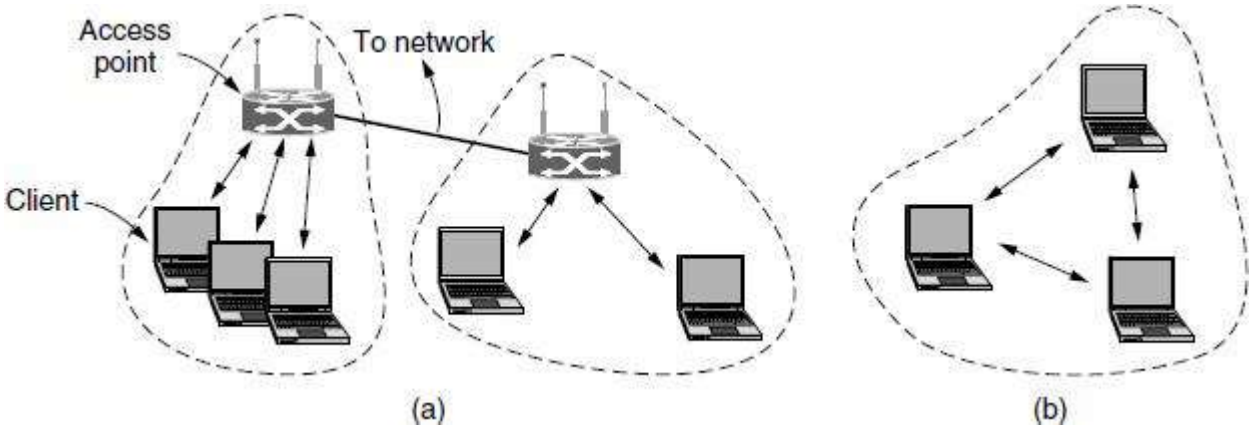
**The 802.11 Architecture and Protocol Stack**

IEEE has defined the specifications for a wireless LAN, called IEEE 802.11, which covers the physical and data link layers.

802.11 networks can be used in two modes.

- The most popular mode is to connect clients, such as laptops and smart phones, to another network, such as a company intranet or the Internet. This mode is shown in Figure (a).

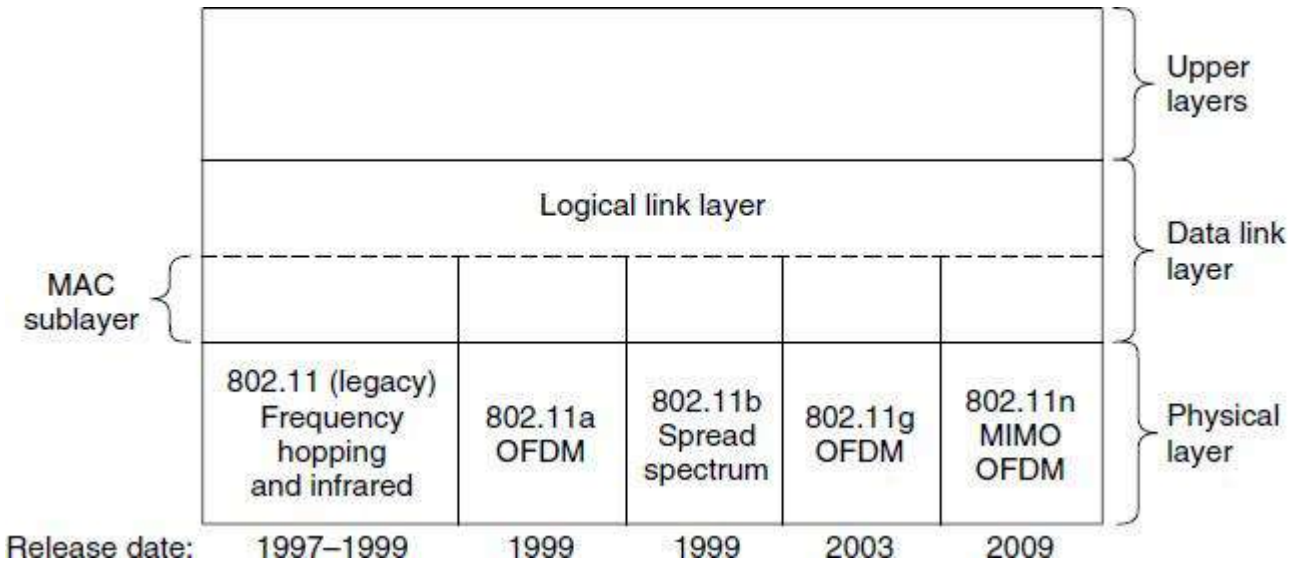
- In infrastructure mode, each client is associated with an **AP (Access Point)** that is in turn connected to the other network. The client sends and receives its packets via the AP.
- Several access points may be connected together, typically by a wired network called a **distribution system**, to form an extended 802.11 network. In this case, clients can send frames to other clients via their APs.




**802.11 architecture. (a) Infrastructure mode. (b) Ad-hoc mode.**

- The other mode, shown in Figure (b), is an **ad hoc network.**
- This mode is a collection of computers that are associated so that they can directly send frames to each other.
- There is no access point.
- Since Internet access is the killer application for wireless, ad hoc networks are not very popular.

All the 802 protocols, including 802.11 and Ethernet, have a certain commonality of structure.




**802.11 protocol stack**

- The stack is the same for clients and APs.
- The physical layer corresponds fairly well to the OSI physical layer, but the data link layer in all the 802 protocols is split into two or more sublayers.
- In 802.11, the **MAC (Medium Access Control) sublayer** determines how the channel is allocated, that is, who gets to transmit next.
- The **LLC (Logical Link Control) sublayer**, whose job it is to hide the differences between the different 802 variants and make them indistinguishable as far as the network layer is concerned.

**802.11 Physical Layer**

All of the 802.11 techniques use short-range radios to transmit signals in either the 2.4-GHz or the 5-GHz ISM frequency bands.

- The first transmission method is **802.11b.** It is a spread-spectrum method that supports rates of 1, 2, 5.5, and 11 Mbps, though in practice the operating rate is nearly always 11 Mbps.
- Spreading is used to satisfy the FCC requirement that power be spread over the ISM band. The spreading sequence used by 201.11b is a **Barker sequence.**
- **802.11a**, which supports rates up to 54 Mbps in the 5-GHz ISM band.
- The **802.11a** method is based on **OFDM (Orthogonal Frequency Division Multiplexing)** because OFDM uses the spectrum efficiently and resists wireless signal degradations such as multipath.
- All wireless communications equipment operating in the ISM bands in the U.S. to use spread spectrum, so it got to work on **802.11g,** which was approved by IEEE in 2003.
- The goal for **802.11n** was throughput of at least 100 Mbps after all the wireless overheads were removed.

| *IEEE* | *Technique* | *Band* | *Modulation* | *Rate (Mbps)* |
|---|---|---|---|---|
| 802.11 | FHSS | 2.4 GHz | FSK | 1 and 2 |
| | DSSS | 2.4 GHz | PSK | 1 and 2 |
| | | Infrared | PPM | 1 and 2 |
| 802.11a | OFDM | 5.725 GHz | PSK or QAM | 6 to 54 |
| 802.11b | DSSS | 2.4 GHz | PSK | 5.5 and 11 |
| 802.11g | OFDM | 2.4 GHz | Different | 22 and 54 |

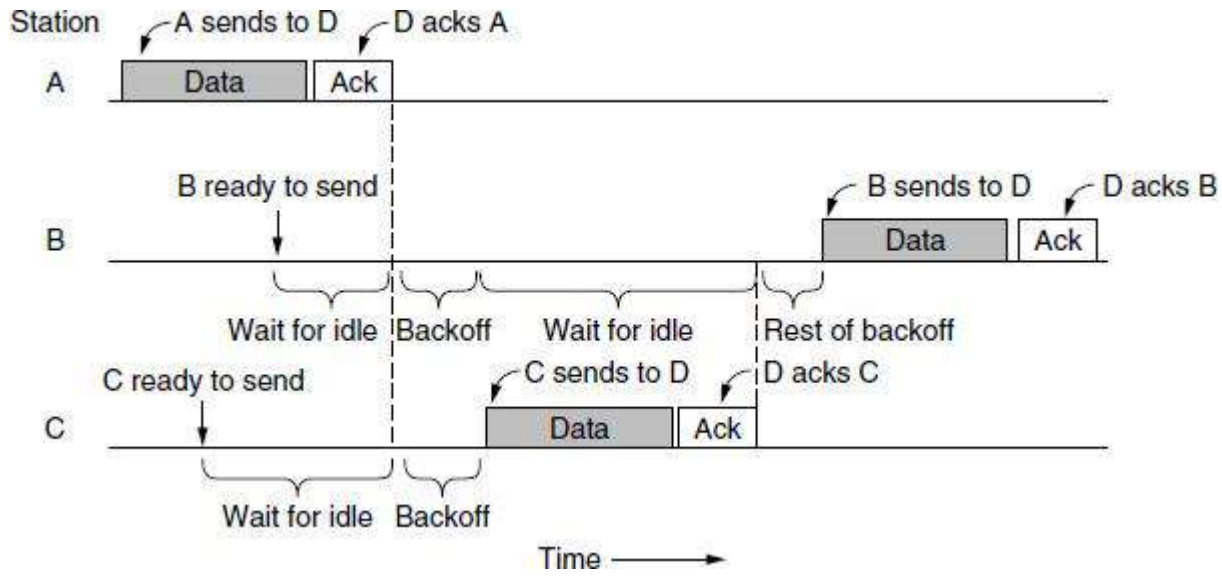
**Physical layers**

**13. Explain MAC sub layer protocol and frame structure of IEEE 802.11? (11 MARKS) (NOV 2016)**

**802.11 MAC Sublayer Protocol**

- 802.11 tries to avoid collisions with a protocol called CSMA/CA (CSMA with Collision Avoidance).
- This protocol is conceptually similar to Ethernet's CSMA/CD, with channel sensing before sending and exponential back off after collisions.
- However, a station that has a frame to send starts with a random backoff.

An example timeline is shown in Figure.




**Sending a frame with CSMA/CA**

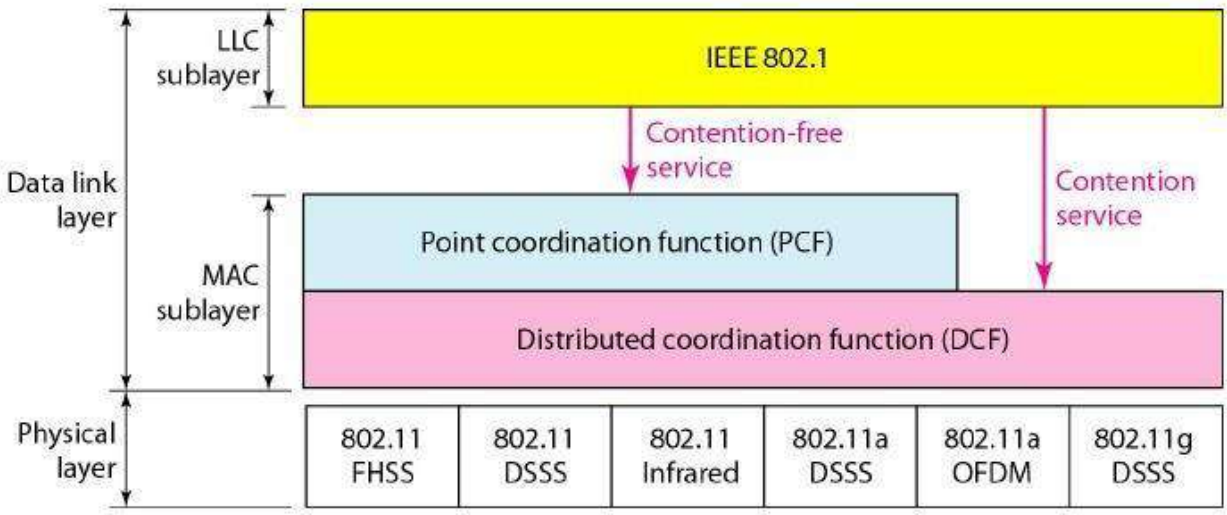
Station A is the first to send a frame. While A is sending, stations B and C become ready to send. The channel is busy and wait for it to become idle. Shortly after A receives an acknowledgement, the channel goes idle. However, rather than sending a frame right away and colliding, B and C both perform a backoff. C picks a short backoff, and thus sends first. B pauses its countdown while it senses that C is using the channel, and resumes after C has received an acknowledgement. B soon completes its backoff and sends its frame.

Compared to Ethernet, there are two main differences.

1. First, starting backoffs early helps to avoid collisions. This avoidance is worthwhile because collisions are expensive, as the entire frame is transmitted even if one occurs.
2. Second, acknowledgements are used to infer collisions because collisions cannot be detected.

IEEE 802.11 defines two MAC sublayers:

1. **Distributed Coordination Function (DCF)** and
2. **Point Coordination Function (PCF)**



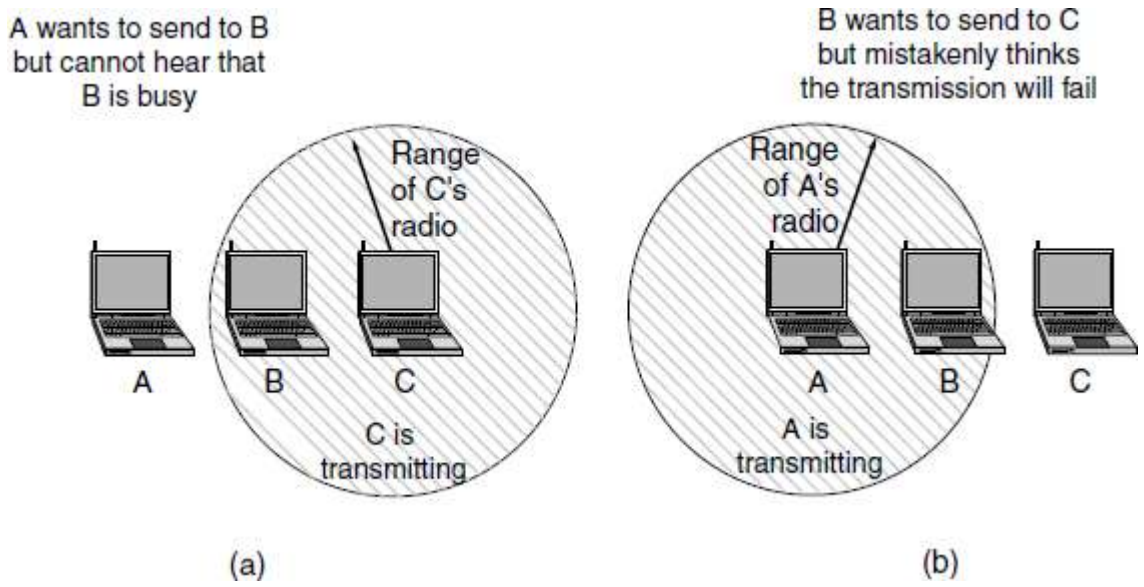

**MAC layers in IEEE 802.11 standard**

- The mode of operation is called **DCF (Distributed Coordination Function)** because each station acts independently, without any kind of central control.
- The standard also includes an optional mode of operation called **PCF (Point Coordination Function)** in which the access point controls all activity in its cell, just like a cellular base station.

The second problem is that the transmission ranges of different stations may be different. With a wire, the system is engineered so that all stations can hear each other. With the complexities of RF propagation this situation does not hold for wireless stations. Consequently, situations such as the hidden terminal problem mentioned earlier and illustrated again in Figure (a).

Since not all stations are within radio range of each other, transmissions going on in one part of a cell may not be received elsewhere in the same cell.

In this example, station C is transmitting to station B. If A senses the channel, it will not hear anything and will falsely conclude that it may now start transmitting to B. This decision leads to a collision.



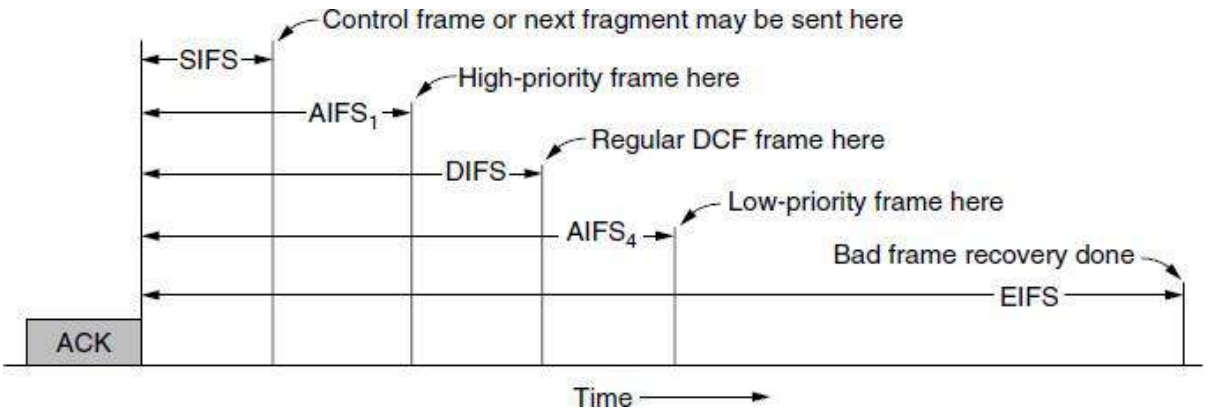

**a) The hidden terminal problem. (b) The exposed terminal problem.**

The inverse situation is the exposed terminal problem, illustrated in Figure (b). Here, B wants to send to C, so it listens to the channel. When it hears a transmission, it falsely concludes that it may not send to C, even though A may in fact be transmitting to D (not shown). This decision wastes a transmission opportunity.

To reduce ambiguities about which station is sending, 802.11 defines channel sensing to consist of both physical sensing and virtual sensing. Physical sensing simply checks the medium to see if there is a valid signal.

With virtual sensing, each station keeps a logical record of when the channel is in use by tracking the **NAV (Network Allocation Vector).** Each frame carries a NAV field that says how long the sequence of which this frame is part will take to complete. Stations that overhear this frame know that the channel will be busy for the period indicated by the NAV, regardless of whether they can sense a physical signal.

1. The interval between regular data frames is called the **DIFS (DCF Inter Frame Spacing).** Any station may attempt to acquire the channel to send a new frame after the medium has been idle for DIFS.
2. The shortest interval is **SIFS (Short Inter Frame Spacing).** It is used to allow the parties in a single dialog the chance to go first.
3. The two **AIFS (Arbitration Inter Frame Space)** intervals show examples of two different priority levels.
4. The last time interval, **EIFS (Extended Inter Frame Spacing),** is used only by a station that has just received a bad or unknown frame, to report the problem.

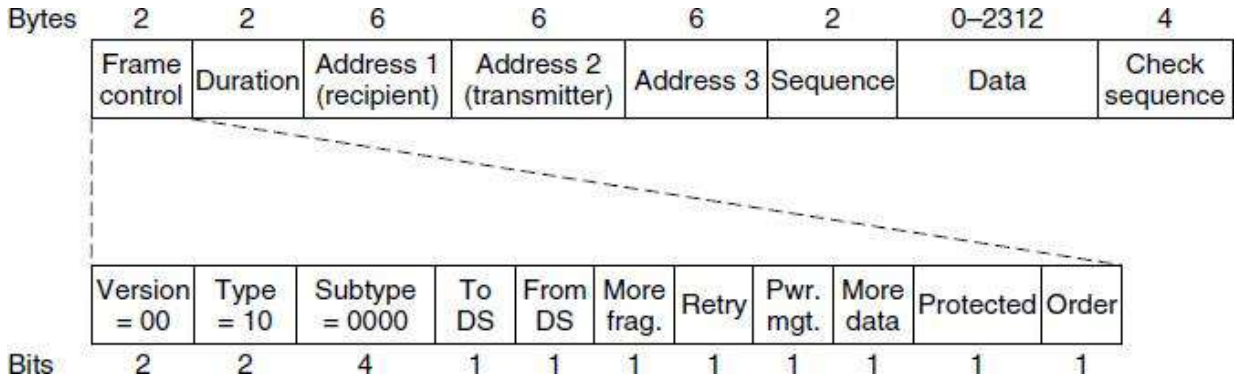



**Inter Frame spacing in 802.11**

## 802.11 Frame Structure

- The 802.11 standard defines three different classes of frames in the air: data, control, and management.
- Each of these has a header with a variety of fields used within the MAC sublayer.

The format of the data frame




**Format of the 802.11 data frame**

- First comes the **Frame control field**, which is made up of 11 subfields.
- The first of these is the **Protocol version**, set to 00. It is there to allow future versions of 802.11 to operate at the same time in the same cell.
- The **Type** (data, control, or management) and **Subtype** fields (e.g., RTS or CTS). For a regular data frame (without quality of service), they are set to 10 and 0000 in binary.
- The **To** DS and **From** DS bits are set to indicate whether the frame is going to or coming from the network connected to the APs, which is called the distribution system.
- The **More fragments** bit means that more fragments will follow.
- The **Retry bit** marks a retransmission of a frame sent earlier.
- The **Power management** bit indicates that the sender is going into power-save mode.
- The **More data** bit indicates that the sender has additional frames for the receiver.
- The **Protected Frame** bit indicates that the frame body has been encrypted for security
- Finally, the **Order bit** tells the receiver that the higher layer expects the sequence of frames to arrive strictly in order.

- The second field of the data frame, the **Duration field**, tells how long the frame and its acknowledgement will occupy the channel, measured in microseconds.
- Data frames sent to or from an AP have three **addresses**, all in standard IEEE 802 format. The first address is the receiver, and the second address is the transmitter.
- The **Sequence field** numbers frames so that duplicates can be detected. Of the 16 bits available, 4 identify the fragment and 12 carry a number that is advanced with each new transmission.
- The **Data field** contains the payload, up to 2312 bytes. The first bytes of this payload are in a format known as **LLC (Logical Link Control).** This layer is the glue that identifies the higher-layer protocol (e.g., IP) to which the payloads should be passed.
- The **Frame check sequence,** which is the same 32-bit CRC.

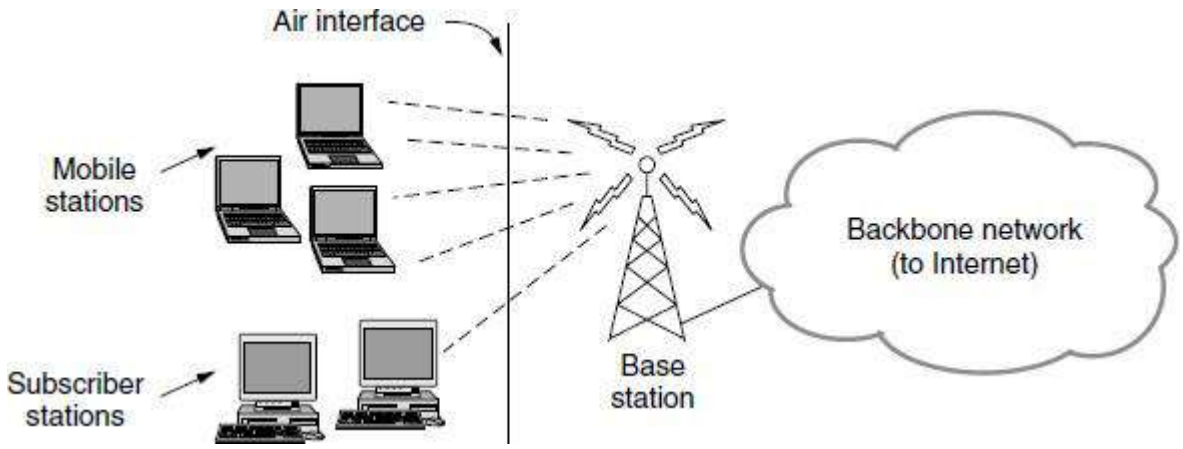
- Management frames have the same format as data frames, plus a format for the data portion that varies with the subtype (e.g., parameters in beacon frames).
- Control frames are short.
- Like all frames, they have the **Frame control, Duration**, and **Frame check sequence** fields.
- However, they may have only one address and no data portion.
- Most of the key information is conveyed with the Subtype field (e.g., ACK, RTS and CTS).

**14. Explain in detail about Wireless LAN 802.16 Protocol architecture? (6 MARKS)**

- The 802.16 architecture is base stations connect directly to the provider's backbone network, which is in turn connected to the Internet.
- The base stations communicate with stations over the wireless air interface.

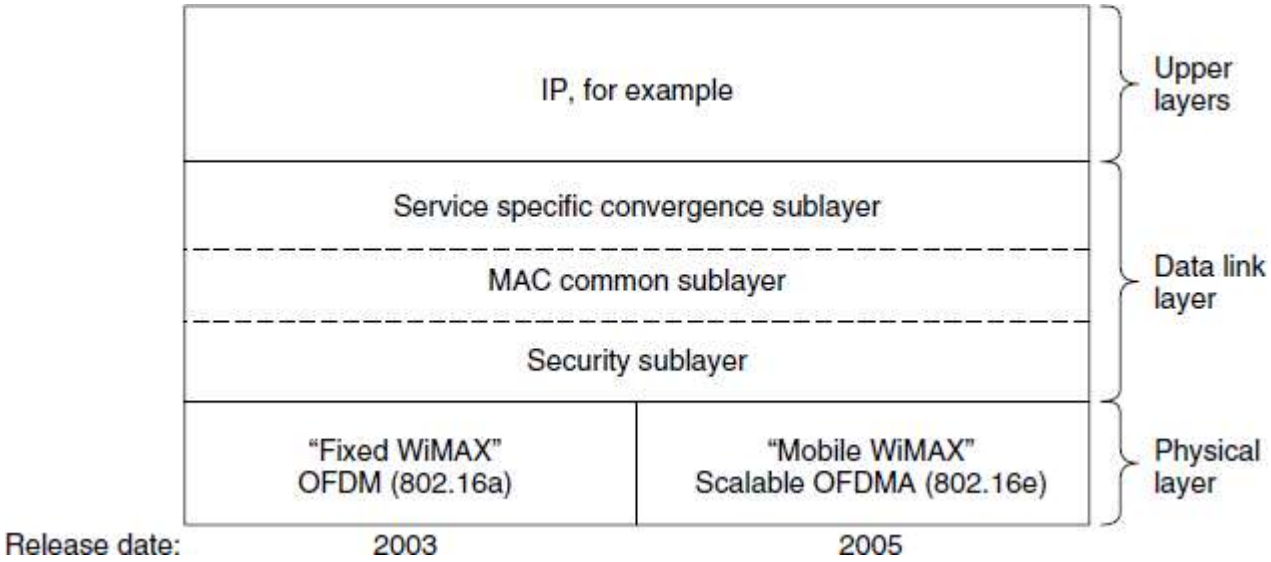
Two kinds of stations are

1. **Subscriber stations** remain in a fixed location, for example, broadband Internet access for homes.
2. **Mobile stations** can receive service while they are moving, for example, a car equipped with WiMAX.




**The 802.16 architecture**

The 802.16 protocol stack that is used across the air interface is shown in Figure.




**The 802.16 protocol stack**

The general structure is similar to that of the other 802 networks, but with more sublayers. The bottom layer deals with transmission, and here we have shown only the popular offerings of 802.16, fixed and mobile WiMAX. There is a different physical layer for each offering. Both layers operate in licensed spectrum below 11 GHz and use OFDM, but in different ways.

Above the physical layer, the data link layer consists of three sublayers. The bottom one deals with privacy and security, which is far more crucial for public outdoor networks than for private indoor networks. It manages encryption, decryption, and key management.

Next comes the MAC common sublayer part. This part is where the main protocols, such as channel management, are located. The model here is that the base station completely controls the system. It can schedule the downlink (i.e., base to subscriber) channels very efficiently and plays a major role in managing the uplink (i.e., subscriber to base) channels as well. The feature of this MAC sublayer is that, unlike those of the other 802 protocols, it is completely connection oriented, in order to provide quality of service guarantees for telephony and multimedia communication.

The service-specific convergence sublayer takes the place of the logical link sublayer in the other 802 protocols. Its function is to provide an interface to the network layer. Different convergence layers are defined to integrate seamlessly with different upper layers.

The important choice is IP, though the standard defines mappings for protocols such as Ethernet and ATM too. Since IP is connectionless and the 802.16 MAC sublayer is connection-oriented, this layer must map between addresses and connections.

**15. Explain about the uses of bridges and learning bridges using Data link layer switching? (6 MARKS)**

**Uses of Bridges**

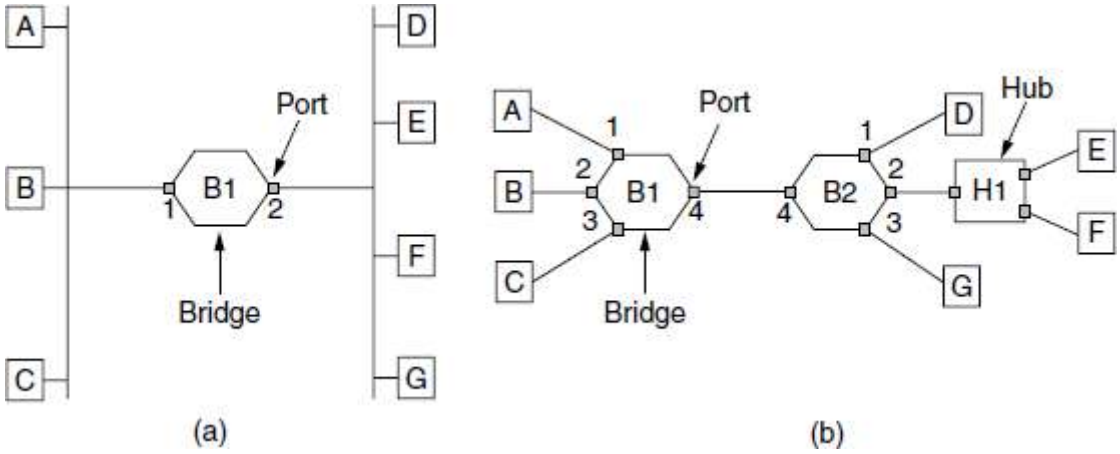
The reasons why a single organization may end up with multiple LANs are

1. First, many university and corporate departments have their own LANs to connect their own personal computers, servers, and devices such as printers. Since the goals of the various departments differ, different departments may set up different LANs, without regard to what other departments are doing.
2. Second, the organization may be geographically spread over several buildings separated by considerable distances. It may be cheaper to have separate LANs in each building and connect them with bridges and a few long-distance fiber optic links than to run all the cables to a single central switch.
3. Third, it may be necessary to split what is logically a single LAN into separate LANs (connected by bridges) to accommodate the load. At many large universities, for example, thousands of workstations are available for student and faculty computing.

To make these benefits easily available, ideally bridges should be completely transparent. It should be possible to go out and buy bridges, plug the LAN cables into the bridges, and have everything work perfectly, instantly. There should be no hardware changes required, no software changes required, no setting of address switches, no downloading of routing tables or parameters, nothing at all. Just plug in the cables and walk away.

**Learning Bridges**

The topology of two LANs bridged together is shown in Figure.




**(a) Bridge connecting two multidrop LANs. (b) Bridges (and a hub) connecting seven point-to-point stations.**

- On the left-hand side, two multi-drop LANs, such as classic Ethernets, are joined by a special station—the bridge—that sits on both LANs.
- On the right-hand side, LANs with point-to-point cables, including one hub, are joined together.
- The bridges are the devices to which the stations and hub are attached.
- If the LAN technology is Ethernet, the bridges are better known as Ethernetswitches.

Bridges were developed when classic Ethernets were in use, so they are often shown in topologies with multi-drop cables, as in Figure (a). However, all the topologies that are encountered today are comprised of point-to-point cables and switches. The bridges work the same way in both settings.

All of the stations attached to the same port on a bridge belong to the same collision domain, and this is different than the collision domain for other ports. If there is more than one station, as in a classic Ethernet, a hub, or a half-duplex link, the CSMA/CD protocol is used to send frames.

To bridge multi-drop LANs, a bridge is added as a new station on each of the multi-drop LANs, as in Figure (a). To bridge point-to-point LANs, the hubs are either connected to a bridge or, preferably, replaced with a bridge to increase performance. In Figure (b), bridges have replaced all but one hub.

The algorithm used by the bridges is **backward learning.**

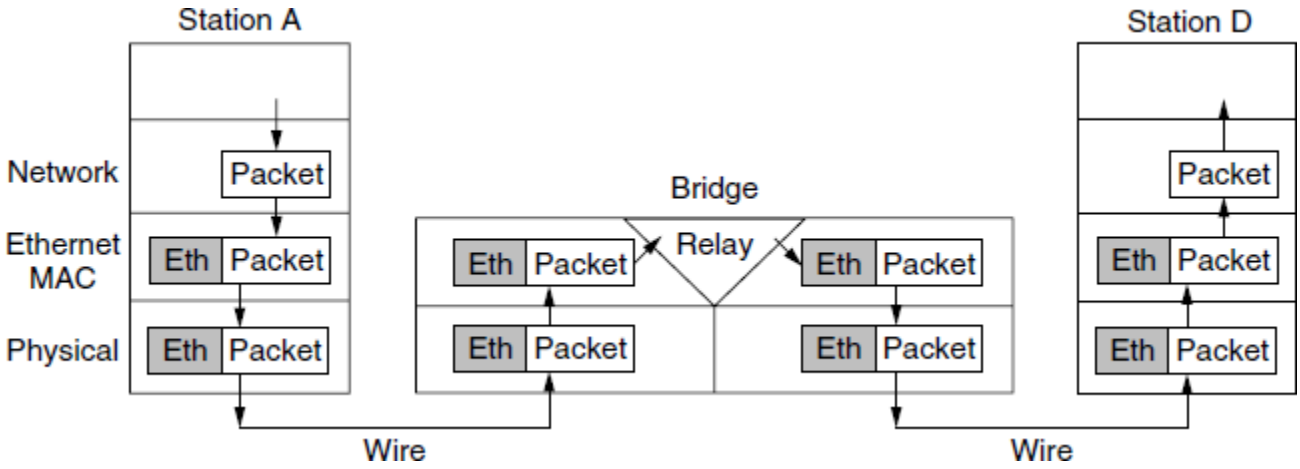
The routing procedure for an incoming frame depends on the port it arrives on (the source port) and the address to which it is destined (the destination address).

The procedure is as follows

1. If the port for the destination address is the same as the source port, discard the frame.
2. If the port for the destination address and the source port are different, forward the frame on to the destination port.
3. If the destination port is unknown, use flooding and send the frame on all ports except the source port.

The design reduces the latency of passing through the bridge, as well as the number of frames that the bridge must be able to buffer. It is referred to as **cut-through switching** or **wormhole routing** and is usually handled in hardware.

The protocol stack view of processing is shown in Figure




**Protocol processing at a bridge**

The packet comes from a higher layer and descends into the Ethernet MAC layer. It acquires an Ethernet header. This unit is passed to the physical layer, goes out over the cable, and is picked up by the bridge.

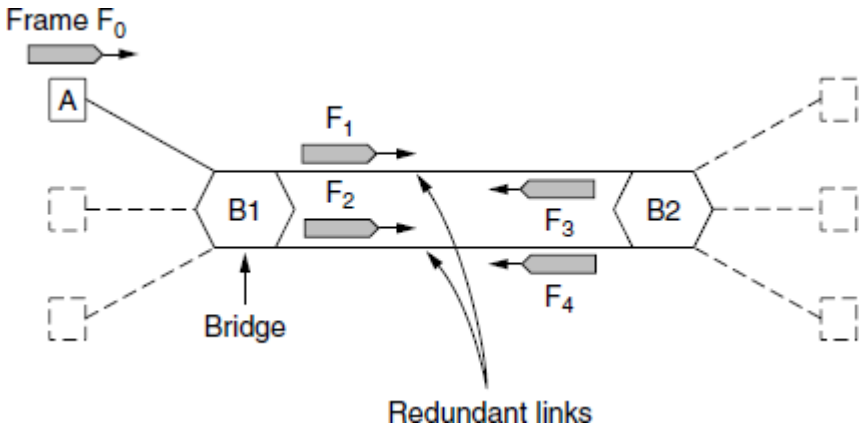
In the bridge, the frame is passed up from the physical layer to the Ethernet MAC layer. This layer has extended processing compared to the Ethernet MAC layer at a station. It passes the frame to a relay, still within the MAC layer.

The bridge relay function uses only the Ethernet MAC header to determine how to handle the frame. In this case, it passes the frame to the Ethernet MAC layer of the port used to reach station D, and the frame continues on its way.

**16. Write a short notes on Spanning Tree Bridges? (5 MARKS) (NOV 2015)**

- In graph theory, a **spanning tree** is a graph in which there is no loop.
- In a bridged LAN, this means creating a topology in which each LAN can be reached from any other LAN through one path only (no loop).
- We cannot change the physical topology of the system because of physical connections between cables and bridges, but we can create a logical topology that overlays the physical one.

- To increase reliability, redundant links can be used between bridges. In the example of Figure, there are two links in parallel between a pair of bridges.
- This design ensures that if one link is cut, the network will not be partitioned into two sets of computers that cannot talk to each other.




**Bridges with two parallel links**

- However, this redundancy introduces some additional problems because it creates loops in the topology.
- An example of these problems can be seen by looking at how a frame sent by A to a previously unobserved destination is handled in Figure.
- Each bridge follows the normal rule for handling unknown destinations, which is to flood the frame. Call the frame from A that reaches bridge B1 frame F0.
- The bridge sends copies of this frame out all of its other ports.
- We will only consider the bridge ports that connect B1 to B2.
- Since there are two links from B1 to B2, two copies of the frame will reach B2. They are shown in Figure as F1 and F2.

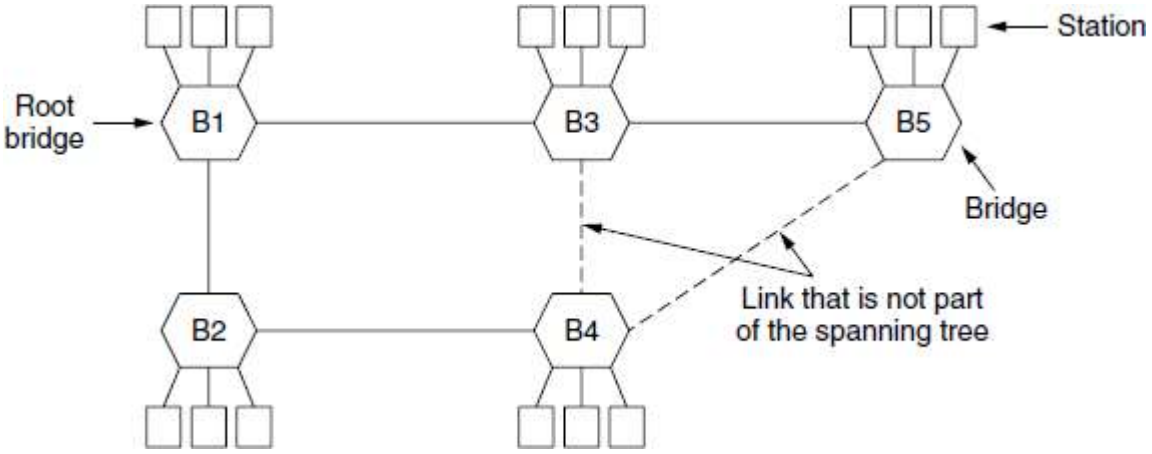
Shortly thereafter, bridge B2 receives these frames. However, it does not (and cannot) know that they are copies of the same frame, rather than two different frames sent one after the other. So bridge B2 takes F1 and sends copies of it out all the other ports, and it also takes F2 and sends copies of it out all the other ports. This produces frames F3 and F4 that are sent along the two links back to B1. Bridge B1 then sees two new frames with unknown destinations and copies them again. This cycle goes on forever.

The solution to this difficulty is for the bridges to communicate with each other and overlay the actual topology with a spanning tree that reaches every bridge. In effect, some potential connections between bridges are ignored in the interest of constructing a fictitious loop-free topology that is a subset of the actual topology.

For example, in Figure we see five bridges that are interconnected and also have stations connected to them. Each station connects to only one bridge. There are some redundant connections between the bridges so that frames will be forwarded in loops if all of the links are used.

This topology can be thought of as a graph in which the bridges are the nodes and the point-to-point links are the edges. The graph can be reduced to a spanning tree, which has no cycles by definition, by dropping the links shown as dashed lines in Figure.

Using this spanning tree, there is exactly one path from every station to every other station. Once the bridges have agreed on the spanning tree, all forwarding between stations follows the spanning tree. Since there is a unique path from each source to each destination, loops are impossible.




**A spanning tree connecting five bridges. The dashed lines are links that are not part of the spanning tree.**

To build the spanning tree, the bridges run a **distributed algorithm**. Each bridge periodically broadcasts a configuration message out all of its ports to its neighbors and processes the messages it receives from other bridges, as described next. These messages are not forwarded, since their purpose is to build the tree, which can then be used for forwarding.

The bridges must first choose one bridge to be the root of the spanning tree. To make this choice, they each include an identifier based on their MAC address in the configuration message, as well as the identifier of the bridge they believe to be the root. MAC addresses are installed by the manufacturer and guaranteed to be unique worldwide, which makes these identifiers convenient and unique. The bridges choose the bridge with the lowest identifier to be the root. After enough messages have been exchanged to spread the news, all bridges will agree on which bridge is the root. In Figure, bridge B1 has the lowest identifier and becomes the root.

Next, a tree of shortest paths from the root to every bridge is constructed. In Figure, bridges B2 and B3 can each be reached from bridge B1 directly, in one hop that is a shortest path. Bridge B4 can be reached in two hops, via either B2 or B3. To break this tie, the path via the bridge with the lowest identifier is chosen, so B4 is reached via B2. Bridge B5 can be reached in two hops via B3.

To find these shortest paths, bridges include the distance from the root in their configuration messages. Each bridge remembers the shortest path it finds to the root. The bridges then turn off ports that are not part of the shortest path.

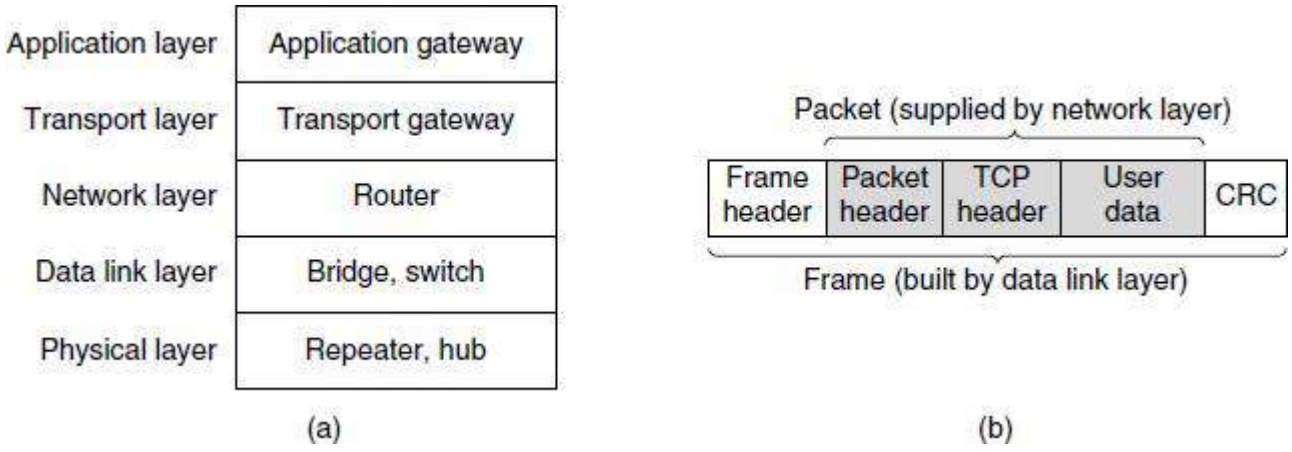
Although the tree spans all the bridges, not all the links (or even bridges) are necessarily present in the tree. This happens because turning off the ports prunes some links from the network to prevent loops. Even after the spanning tree has been established, the algorithm continues to run during normal operation to automatically detect topology changes and update the tree.

**17. Explain in detail about Repeaters, Hubs, Bridges, Switches, Routers, and Gateways? (11 MARKS) (NOV 2015)**

We divide connecting devices into six different categories based on the layer in which they operate in a network.

1. Repeaters,
2. Hubs,
3. Bridges,
4. Switches,
5. Routers, and
6. Gateways

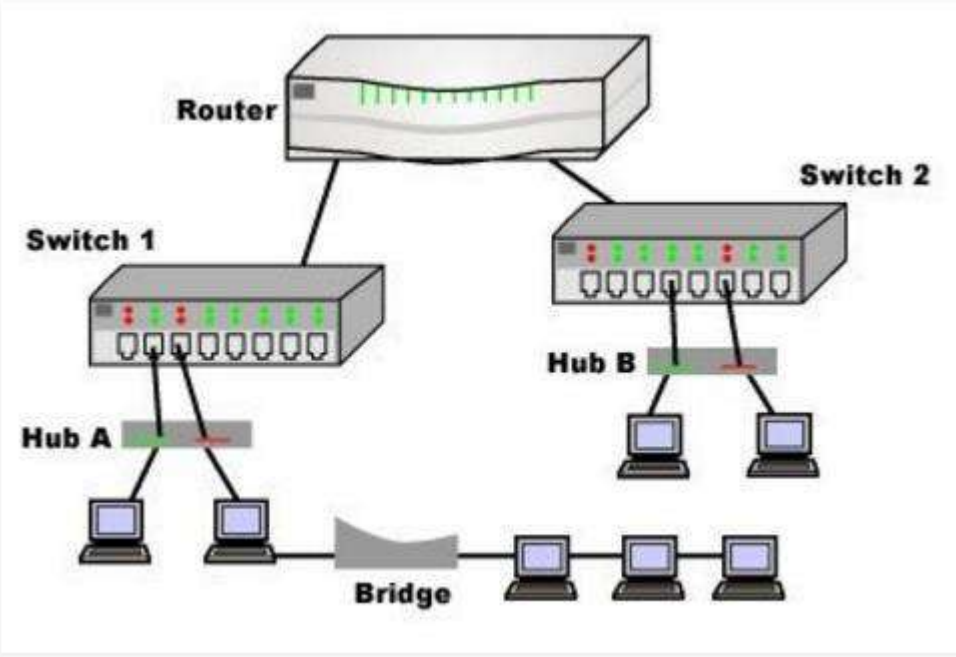
The key to understanding these devices is to realize that they operate in different layers,




**(a) Which device is in which layer. (b) Frames, packets, and headers.**

The layer matters because different devices use different pieces of information to decide how to switch. In a typical scenario, the user generates some data to be sent to a remote machine. Those data are passed to the transport layer, which then adds a header (for example, a TCP header) and passes the resulting unit down to the network layer. The network layer adds its own header to form a network layer packet. The network layer adds its own header to form a network layer packet (e.g., an IP packet).

In Figure(b), we see the IP packet shaded in gray. Then the packet goes to the data link layer, which adds its own header and checksum (CRC) and gives the resulting frame to the physical layer for transmission, for example, over a LAN.




**Repeaters, Hubs, Bridges, Switches, Routers, and Gateways**

**Repeaters**

- The switching devices and see how they relate to the packets and frames.
- At the bottom, in the physical layer, we find the **repeaters**.
- These are analog devices that work with signals on the cables to which they are connected.
- A signal appearing on one cable is cleaned up, amplified, and put out on another cable.
- Repeaters do not understand frames, packets, or headers.
- They understand the symbols that encode bits as volts.
- Classic Ethernet, for example, was designed to allow four repeaters that would boost the signal to extend the maximum cable length from 500 meters to 2500 meters.

**Repeater** operates at the physical layer. Its job is to regenerate the signal over the same network before the signal becomes too weak or corrupted so as to extend the length to which the signal can be transmitted over the same network. An important point to be noted about repeaters is that they do no amplify the signal. When the signal becomes weak, they copy the signal bit by bit and regenerate it at the original strength. It is a 2 port device.

**Hubs**

- A **hub** has a number of input lines that it joins electrically.
- Frames arriving on any of the lines are sent out on all the others.
- If two frames arrive at the same time, they will collide, just as on a coaxial cable.
- All the lines coming into a hub must operate at the same speed.

- Hubs differ from repeaters in that they do not (usually) amplify the incoming signals and are designed for multiple input lines, but the differences are slight.

A **hub** is basically a multiport repeater. A hub connects multiple wires coming from different branches, for example, the connector in star topology which connects different stations. Hubs cannot filter data, so data packets are sent to all connected devices. In other words, collision domain of all hosts connected through Hub remains one. Also, they do not have intelligence to find out best path for data packets which leads to inefficiencies and wastage.

**Bridges**

- A bridge connects two or more LANs. Like a hub, a modern bridge has multiple ports, usually enough for 4 to 48 input lines of a certain type.
- When a frame arrives, the bridge extracts the destination address from the frame header and looks it up in a table to see where to send the frame.
- Bridges offer much better performance than hubs, and the isolation between bridge ports also means that the input lines may run at different speeds, possibly even with different network types.
- A common example is a bridge with ports that connect to 10-, 100-, and 1000-Mbps Ethernet.
- Buffering within the bridge is needed to accept a frame on one port and transmit the frame out on a different port.
- If frames come in faster than they can be retransmitted, the bridge may run out of buffer space and have to start discarding frames.
- For example, if a gigabit Ethernet is pouring bits into a 10-Mbps Ethernet at top speed, the bridge will have to buffer them, hoping not to run out of memory.

A **bridge** operates at data link layer. A bridge is a repeater, with add on functionality of filtering content by reading the MAC addresses of source and destination. It is also used for interconnecting two LANs working on the same protocol. It has a single input and single output port, thus making it a 2 port device.

**Switches**

- A **switch** may refer to an Ethernet switch or a completely different kind of device that makes forwarding decisions, such as a telephone switch. A **switch** is a multi-port bridge with a buffer and a design that can boost its efficiency (large number of ports imply less traffic) and performance. Switch is data link layer device. Switch can perform error checking before forwarding data that makes it very efficient as it does not forward packets that have errors and forward good packets selectively to correct port only. In other words, switch divides collision domain of hosts, but broadcast domain remains same.

**Routers**

- When a packet comes into a **router,** the frame header and trailer are stripped off and the packet located in the frame's payload field is passed to the routing software.

- This software uses the packet header to choose an output line.
- For an IP packet, the packet header will contain a 32-bit (IPv4) or 128-bit (IPv6) address, but not a 48-bit IEEE 802 address.

A **router** is a device like a switch that routes data packets based on their IP addresses. Router is mainly a Network Layer device. Routers normally connect LANs and WANs together and have a dynamically updating routing table based on which they make decisions on routing the data packets. Router divide broadcast domains of hosts connected through it.

**Gateways**

- Up another layer, we find **transport gateways**. These connect two computers that use different connection-oriented transport protocols. For example, suppose a computer using the connection-oriented TCP/IP protocol needs to talk to a computer using a different connection-oriented transport protocol called SCTP. The transport gateway can copy the packets from one connection to the other.
- Finally, **application gateways** understand the format and contents of the data and can translate messages from one format to another. An email gateway could translate Internet messages into SMS messages for mobile phones.

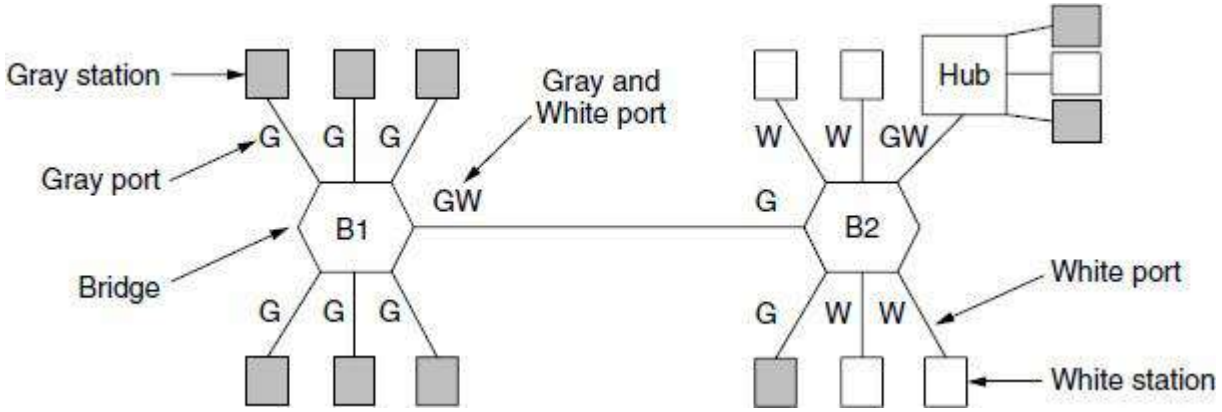
A **gateway** is a passage to connect two networks together that may work upon different networking models. They basically works as the messenger agents that take data from one system, interpret it, and transfer it to another system. Gateways are also called protocol converters and can operate at any network layer. Gateways are generally more complex than switch or router.

**18. Explain in detail about Virtual LANs? (6 MARKS)**

- Virtual Local Area Network (VLAN) as a local area network configured by software, not by physical wiring.
- VLAN (Virtual LAN) has been standardized by the IEEE 802 committee and is now widely deployed in many organizations.

VLANs are based on VLAN-aware switches. To set up a VLAN-based network, the network administrator decides how many VLANs there will be, which computers will be on which VLAN, and what the VLANs will be called. Often the VLANs are (informally) named by colors, since it is then possible to print color diagrams showing the physical layout of the machines, with the members of the red LAN in red, members of the green LAN in green, and so on. In this way, both the physical and logical layouts are visible in a single view.

As an example, consider the bridged LAN of Figure, in which nine of the machines belong to the G (gray) VLAN and five belong to the W (white) VLAN. Machines from the gray VLAN are spread across two switches, including two machines that connect to a switch via a hub.




**Two VLANs, gray and white, on a bridged LAN**

To make the VLANs function correctly, configuration tables have to be set up in the bridges. These tables tell which VLANs are accessible via which ports. When a frame comes in from, say, the gray VLAN, it must be forwarded on all the ports marked with a G. This holds for ordinary (i.e., unicast) traffic for which the bridges have not learned the location of the destination, as well as for multicast and broadcast traffic.

As an example, suppose that one of the gray stations plugged into bridge B1 in Figure sends a frame to a destination that has not been observed beforehand. Bridge B1 will receive the frame and see that it came from a machine on the gray VLAN, so it will flood the frame on all ports labeled G (except the incoming port).

The frame will be sent to the five other gray stations attached to B1 as well as over the link from B1 to bridge B2. At bridge B2, the frame is similarly forwarded on all ports labeled G. This sends the frame to one further station and the hub (which will transmit the frame to all of its stations). The hub has both labels because it connects to machines from both VLANs. The frame is not sent on other ports without G in the label because the bridge knows that there are no machines on the gray VLAN that can be reached via these ports.

In our example, the frame is only sent from bridge B1 to bridge B2 because there are machines on the gray VLAN that are connected to B2. Looking at the white VLAN, we can see that the bridge B2 port that connects to bridge B1 is not labeled W. This means that a frame on the white VLAN will not be forwarded from bridge B2 to bridge B1. This behavior is correct because no stations on the white VLAN are connected to B1.

## PONDICHERRY UNIVERSITY QUESTIONS

### 2 MARKS

1. What are the difference between collision free and contention based network protocols? **(NOV 2015) (Ref.Qn.No.41, Pg.no.8)**
2. What are virtual LANs? **(NOV 2015) (Ref.Qn.No.70, Pg.no.13)**
3. Define hamming distance. **(MAY 2016) (Ref.Qn.No.17, Pg.no.4)**
4. What is hub? **(MAY 2016) (Ref.Qn.No.65, Pg.no.12)**
5. Define spanning tree. **(MAY 2016) (Ref.Qn.No.63, Pg.no.12)**
6. Write the importance of CRC in the network. **(NOV 2016) (Ref.Qn.No.25, Pg.no.5)**
7. Differentiate between hubs and switches. **(NOV 2016) (Ref.Qn.No.65, 67, Pg.no.12)**
8. Differentiate between physical and logical address. **(MAY 2017) (Ref.Qn.No.71, Pg.no.13)**
9. What is the necessity of flow control? **(MAY 2017) (Ref.Qn.No.10, Pg.no.3)**

### 11 MARKS

**NOV 2015 (REGULAR)**

1. (a). Briefly discuss the difference between Hubs, Bridges switches and routers. (6) **(Ref.Qn.No.17, Pg.no.76)**

   (b). Briefly discuss how the CSMA media access protocol works. (5) **(Ref.Qn.No.9, Pg.no.54)**

   **(OR)**

2. (a). Discuss error correction in the link layer protocols. (6) **(Ref.Qn.No.2, Pg.no.20)**

   (b). Briefly explain how spanning tree bridge work. (5) **(Ref.Qn.No.16, Pg.no.73)**

**MAY 2016 (ARREAR)**

1. Explain Error detection and correction briefly. **(Ref.Qn.No.2, Pg.no.20)**

   **(OR)**

2. Describe briefly about CSMA protocol with a diagram. **(Ref.Qn.No.9, Pg.no.54)**

**NOV 2016 (REGULAR)**

1. Explain MAC sub layer protocol and frame structure of IEEE 802.11. **(Ref.Qn.No.13, Pg.no.64)**

   **(OR)**

2. Write short notes on: **(Ref.Qn.No.6, Pg.no.39)**

   (a) Go Back N-ARQ

   (b) Selective Repeat ARQ.

**MAY 2017 (ARREAR)**

1. Describe the CRC code for error detection and hamming code method for error correction of transmitted data units. **(Ref.Qn.No.2, Pg.no.20)**

**(OR)**

2. Describe the purpose and design of stop and wait protocol with both the sender and receiver side algorithm. **(Ref.Qn.No.5, Pg.no.34)**